

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Subject Name: **DATABASE MANAGEMENT SYSTEMS**

Subject Code: **CS T53**

UNIT II

Database Design and the ER Model: Overview of the Design Process – The Entity – Relational Model – Constraints – Removing Redundant Attributes – Entity – Relationship Design Issues – Extended E-R Features – Alternative Notations for Modeling data – Other Aspects of Database Design – Storage and File structure – Indexing and Hashing – Basic Concepts – Ordered Indices – B+ - Tree Index Files – Static Hashing – Dynamic Hashing – Comparison of Ordered Indexing and Hashing – Bitmap Indices – Index Definition in SQL.

Unit II- Two Marks

1. What is an entity relationship model? [DEC 2016][NOV 2017]

The entity relationship model is a collection of basic objects called entities and relationship among those objects. An entity is a thing or object in the real world that is distinguishable from other objects.

2. What are attributes? Give examples. (DEC 2018) (MAY 2019)

An entity is represented by a set of attributes. Attributes are descriptive properties possessed by each member of an entity set.

Example: possible attributes of customer entity are customer name, customer id, customer street, customer city.

3. What is relationship? Give examples

A relationship is an association among several entities.

Example: A depositor relationship associates a customer with each account that he/she has.

4. Define Entity, Entity Set, and extensions of entity set. Give one example for each. (NOV 2012) (DEC 2018)

- Entity – object or thing in the real world. Egs., each person, book, etc.
- Entity set – set of entities of the same entity that share the same properties or attributes. Eg., Customer, loan, account, etc

- Extensions of entity set – individual entities that constitute a set. Eg., individual bank customers

5. Define and give examples to illustrate the four types of attributes in database. (NOV 2010)

- Simple and Composite attribute
 - Simple (address)
 - Composite (street, city, district)
- Single-valued and multi-valued attribute
 - Single-valued (roll no)
 - Multi-valued (colors = {R,B,G})
- Null attribute
 - No values for attribute
- Derived attribute
 - Age derived from Birth date

6. Define the terms

i) Entity type & ii) Entity set

Entity type: An entity type defines a collection of entities that have the same attributes.

Entity set: The set of all entities of the same type is termed as an entity set.

7. What is meant by the degree of relationship set?

The degree of relationship type is the number of participating entity types.

8. Define the terms

i) Key attribute

ii) Value set

Key attribute: An entity type usually has an attribute whose values are distinct from each individual entity in the collection. Such an attribute is called a key attribute.

Value set: Each simple attribute of an entity type is associated with a value set that specifies the set of values that may be assigned to that attribute for each individual entity.

9. Define relationship and participation.

- Relationship is an association among several entities.
- Association between entity sets is referred as participation.

10. Define mapping cardinality or cardinality ratio.

Mapping cardinality or cardinality ratio is the way to express the number of entities to which another entity can be associated through a relationship set. These are most useful in describing binary relationship sets.

11. Explain the four types of mapping cardinality with example.

For a binary relationship set R between entity sets A and B, the mapping cardinality must be one of the following: (Draw the diagrams also)

- One-to-one

An entity in A is associated with at most one entity in B and an entity in B is associated with at most one entity in A. Eg., Roll no entity in Student info entity set and marks details entity set.

- One-to-many

An entity in A is associated with any number of entities in B. But an entity in B is associated with at most one entity in A. Eg., one customer with many loans.

- Many-to-one

An entity in A is associated with at most one entity in B. But an entity in B can be associated with any number of entities in A. Eg., street and city associated to a single person.

- Many-to-many

An entity in A is associated with any number of entities in B. But an entity in B can be associated with any number of entities in A. Eg., same loan by several business partners.

12. Differentiate total participation and partial participation. (i.e., write Definition and Example with illustration)

- Total – participation of an entity set, E in relationship, R is total if every entity in E participates in at least one relationship in R. Eg., loan entity set.
- Partial – if only some entities in E participate in R. Eg., payment weak entity set.

13. Define E-R diagram.

Overall structure of a database can be expressed graphically by E-R diagram for simplicity and clarity.

14. Define weak Entity set. Give an example and explain why it is weak entity set.

Entity set with no sufficient attributes to form a primary key.

- Payment entity set is weak since duplication exists

15. Define discriminator or partial key of a weak entity set. Give example.

Set of attributes that allow distinction to be made among all those entities in the entity set that depend on one particular strong entity. Eg., payment_no in payment entity set. It is also called as partial key.

16. Define strong and weak entity sets? **NOV 2014**

An entity set may not have sufficient attributes to form a primary key. Such an entity set is termed a **weak entity set**. An entity set that has a primary key is termed a **strong entity set**.

17. Define the term Canonical Form? **APR 2014**

Sets of functional dependencies may have redundant dependencies that can be inferred from the others

- For example: $A \rightarrow C$ is redundant in: $\{A \rightarrow B, B \rightarrow C\}$
- Parts of a functional dependency may be redundant
- E.g.: on RHS: $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
- E.g.: on LHS: $\{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$

18.What is E-R modeling? APR 2015

Entity-relationship model (ER model) is a data model for describing the data or information aspects of a business domain or its process requirements, in an abstract way that lends itself to ultimately being implemented in a database such as a relational database. The main components of ER models are entities (things) and the relationships that can exist among them.

19.What is logical data independence?

Logical data is data about database, that is, it stores information about how data is managed inside. For example, a table (relation) stored in the database and all its constraints, applied on that relation.

Logical data independence is a kind of mechanism, which liberalizes itself from actual data stored on the disk. If any changes on table format, it should not change the data residing on the disk.

20. What are the types of indices? (April 2014)

Ordered indices: Based on a sorted ordering of the values.

Hash indices: Based on a uniform distribution of values across a range of bucket. The bucket to which a value is assigned is determined by a function, called a hash function.

21. What are the techniques of indexing & hashing?(Nov 2014) (DEC 2018)

- Access types
- Access time
- Insertion time
- Deletion time
- Space overhead

22.What is dense index?

An index record appears for every search key value in the file. In a dense clustering index, the index record contains search key value and a pointer to the first data record with that search key value.

23.What is sparse index?

An index record appears for only some of the search key values in a file. Such a

24.What is a multilevel index?

Indices with two or more levels are called multilevel indices, multilevel indices are closely related to the tree structures such as binary tree used for memory indexing.

25.What are B+ tree index files?

A B+ tree index takes the form of a balanced tree in which every path from the root of the tree to the leaf of the tree is of the same length.

26.What is the use of static hashing?

To access a file in sequential organization binary search or indexing is used, this creates more i/o operations to avoid this static hashing is used.

27.What is a hash function?

A hash function distributes the stored key uniformly across all the buckets, so that every bucket has the same number of records.

28.What is skew?

Some buckets are assigned more records than the others, so a bucket may overflow even when other buckets still have space. This situation is called **bucket skew**

29.What are the two reasons for the occurrence of skew?

- Multiple records may have the same search key
- The chosen hash function may result in non uniform distribution of search keys

30.What is a hash index?

A hash index organizes the search keys, with their associated pointers into a hash file structure.

31.Define dynamic hashing?

Dynamic hashing techniques allow the hash function to be modified dynamically to accommodate the growth or shrinkage of the database, one form of the dynamic hashing is called **extendable hashing**.

32.What is a bitmap index?

Bitmap indices are a specialized type of index designed for easy querying on multiple keys, although each bitmap index is built on a single key.

33.Define covering indices?

Covering indices are the indices that store the values of some attributes along with the pointers to the record. Storing extra attribute values is useful with secondary indices, since they allow us to answer some queries using just the index, without even looking up the actual records.

34.What is Hashing? (April 2013) (MAY 2019)

Hashing is the transformation of a string of characters into a usually shorter fixed-length value or key that represents the original string. Hashing is used to index and retrieve items in a database because it is faster to find the item using the shorter hashed key than to find it using the original value. It is also used in many encryption algorithms.

35.List out the different types of indices.(April 2011),NOV 2015

Ordered indices: Based on a sorted ordering of the values.

Hash indices: Based on a uniform distribution of values across a range of bucket. The bucket to which a value is assigned is determined by a function, called a hash function

36.What is B+ tree? (APRIL 2015)

A dynamic data structure which adjusts efficiently under inserts and deletes. data entries are stored at the leaf level. automatically reorganizes itself with small, local, changes, in the face of insertions and deletions. Reorganization of entire file is not required to maintain performance.

37. Give example for Composite Attribute. (NOV 2015)

Abdul Kadhira Shahid is the name of a student. Here we can combine three names to a single value.

38. How primary indices differ from secondary indices. (NOV 2015)

Primary index:

A primary index is an index on a set of fields that includes the unique primary key for the field and is guaranteed not to contain duplicates. Also called a **Clustered index**. eg. Employee ID can be an example of it.

Secondary index:

A Secondary index is an index that is not a primary index and may have duplicates. eg. Employee name can be an example of it. Because Employee name can have similar values.

39. Define Referential Integrity [NOV 2017]

Referential integrity in a relational database is consistency between coupled tables. Referential integrity is usually enforced by the combination of a primary key or candidate key (alternate key) and a foreign key. For referential integrity to hold, any field in a table that is declared a foreign key can contain only values from a parent table's primary key or a candidate key.

1. Explain in detail about Entity Set? (11 Marks)

An **entity** is a “thing” or “object” in the real world that is distinguishable from all other objects. For example, each person in an enterprise is an entity. An entity has a set of properties, and the values for some set of properties may uniquely identify an entity. An entity may be concrete, such as a person or a book, or it may be abstract, such as a loan, or a holiday, or a concept.

An **entity set** is a set of entities of the same type that share the same properties, or attributes. The set of all persons who are customers at a given bank, for example, can be defined as the entity set customer. An entity is represented by a set of **attributes**. Attributes are descriptive properties possessed by each member of an entity set. The designation of an attribute for an entity set expresses that the database stores similar information concerning each entity in the entity set; however, each entity may have its own value for each attribute. Possible attributes of the customer entity set are customer-id, customer-name, Customer Street, and customer-city. In real life, there would be further attributes, such as street number, apartment number, state, postal code, and country etc.

Each entity has a **value** for each of its attributes. For instance, a particular customer entity may have the value 321-12-3123 for customer-id, the value Jones for customer name, the value Main for customer-street, and the value Harrison for customer-city. The customer-id attribute is used to uniquely identify customers, since there may be more than one customer with the same name. For each attribute, there is a set of permitted values, called the **domain**, or **value set**, of that attribute. The domain of attribute customer-name might be the set of all text strings of a certain length.

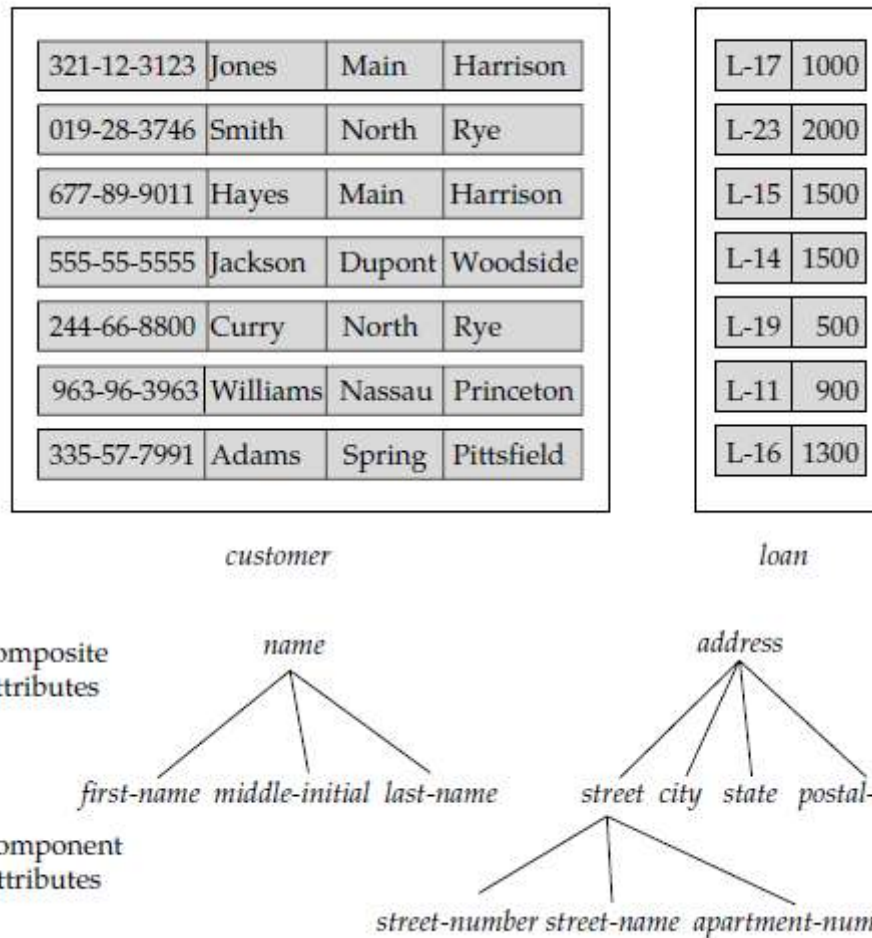
A database thus includes a collection of entity sets, each of which contains any number of entities of the same type. An attribute, as used in the E-R model, can be characterized by the following attribute

types:

- **Simple** and **composite** attributes. In our examples thus far, the attributes have been simple; that is, they are not divided into subparts. **Composite** attributes, on the other hand, can be divided into subparts (that is, other attributes). For example, an attribute name could be structured as a composite attribute consisting of first-name, middle-initial, and last-name.
- **Single-valued** and **multivalued** attributes. The attributes in our examples all have a single value for a particular entity. For instance, the loan-number attribute for a specific loan entity refers to only one loan number. Such attribute are said to be **single valued**.
- An employee may have zero, one, or several phone numbers, and different employees may have different numbers of phones. This type of attribute is said to be **multivalued**.

- **Derived** attribute. The value for this type of attribute can be derived from the values of other related attributes or entities. For instance, let us say that the customer entity set has an attribute loans-held, which represents how many loans a customer has from the bank. We can derive the value for this attribute by counting the number of loan entities associated with that customer.

An attribute takes a **null** value when an entity does not have a value for it. The null value may indicate “not applicable”—that is, that the value does not exist for the entity. For example, one may have no middle name. Null can also designate that an attribute value is unknown. An unknown value may be either missing (the value does exist, but we do not have that information) or not known (we do not know whether or not the value actually exists).



2. Explain relationship set? (11 Marks)

A **relationship** is an association among several entities. For example, we can define a relationship that associates customer Hayes with loan L-15. This relationship specifies that Hayes is a customer with loan number L-15.

A **relationship set** is a set of relationships of the same type. Formally, it is a mathematical relation on $n \geq 2$ (possibly nondistinct) entity sets. If $E_1, E_2, \dots, E_n$ are entity sets, then a relationship set R is a subset of

$$\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$$

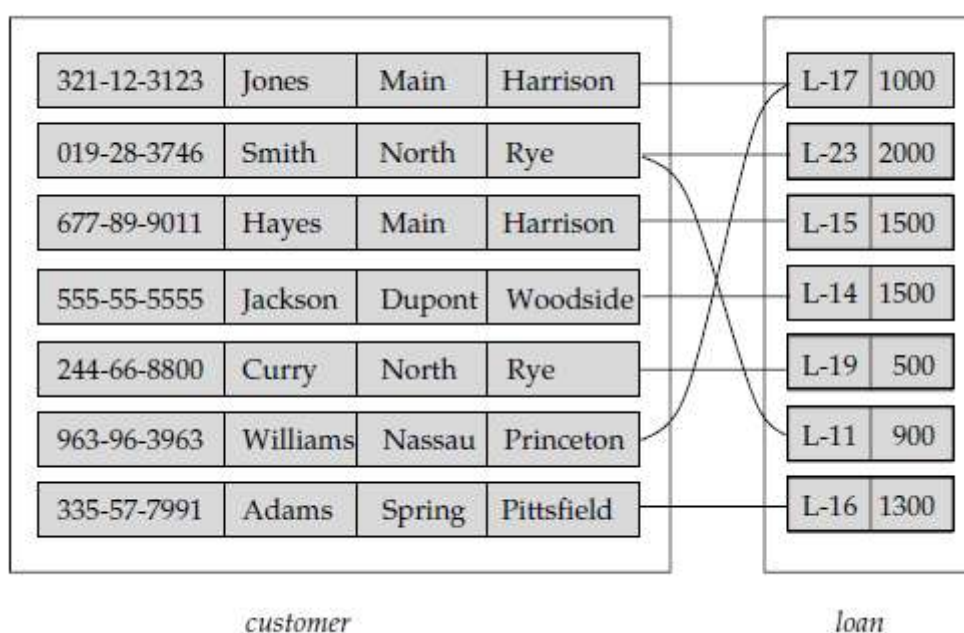
where $(e_1, e_2, \dots, e_n)$ is a relationship.

As another example, consider the two entity sets loan and branch. We can define the relationship set loan-branch to denote the association between a bank loan and the branch in which that loan is maintained.

The association between entity sets is referred to as participation; that is, the entity sets $E_1, E_2, \dots, E_n$ **participate** in relationship set R . A **relationship instance** in an E-R schema represents an association between the named entities.

The function that an entity plays in a relationship is called that entity's **role**. Since entity sets participating in a relationship set are generally distinct, roles are implicit and are not usually specified. The same entity set participates in a relationship set more than once, in different roles. In this type of relationship set, sometimes called a **recursive** relationship set, explicit role names are necessary to specify how an entity participates in a relationship instance. A relationship may also have attributes called **descriptive attributes**. Consider a relationship set depositor with entity sets customer and account. However, there can be more than one relationship set involving the same entity sets. In our example the customer and loan entity sets participate in the relationship set borrower. Additionally, suppose each loan must have another customer who serves as a guarantor for the loan. Then the customer and loan entity sets may participate in another relationship set, guarantor.

The relationship sets borrower and loan-branch provide an example of a **binary** relationship set that is, one that involves two entity sets. Most of the relationship sets in a database system are binary. The number of entity sets that participate in a relationship set is also the **degree** of the relationship set. A binary relationship set is of degree 2; a ternary relationship set is of degree 3.



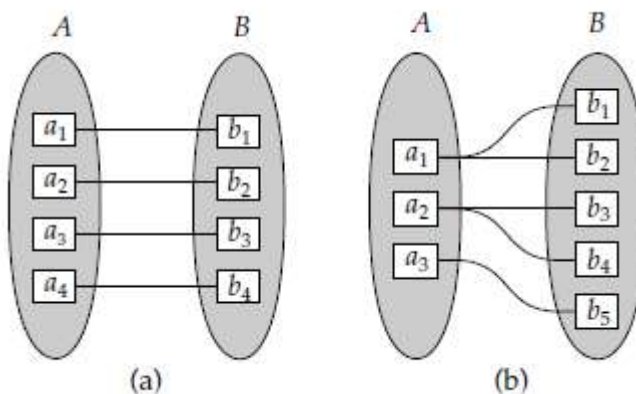
3.Explain in detail about constraints and keys? (11 Marks)

a) Mapping Cardinalities [TWO MARKS NOV 2012]

Mapping cardinalities, or cardinality ratios, express the number of entities to which another entity can be associated via a relationship set. Mapping cardinalities are most useful in describing binary relationship sets, although they can contribute to the description of relationship sets that involve more than two entity sets. In this section, we shall concentrate on only binary relationship sets.

For a binary relationship set R between entity sets A and B , the mapping cardinality must be one of the following:

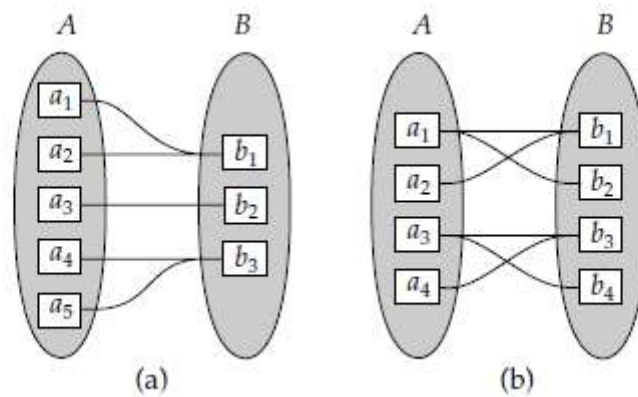
- **One to one.** An entity in A is associated with at most one entity in B , and an entity in B is associated with at most one entity in A . (See Figure 2.4a.)
- **One to many.** An entity in A is associated with any number (zero or more) of entities in B . An entity in B , however, can be associated with at most one entity in A . (See Figure 2.4b.)
- **Many to one.** An entity in A is associated with at most one entity in B . An entity in B , however, can be associated with any number (zero or more) of entities in A . (See Figure 2.5a.)
- **Many to many.** An entity in A is associated with any number (zero or more) of entities in B , and an entity in B is associated with any number (zero or more) of entities in A . (See Figure 2.5b.)



b) Participation Constraints

The participation of an entity set E in a relationship set R is said to be **total** if every entity in E participates in at least one relationship in R . If only some entities in E participate in relationships in R , the participation of entity set E in relationship R is said to be **partial**. For example, we expect every loan entity to be related to at least one customer through the borrower relationship. Therefore the participation of loan in the relationship set borrower is total. In contrast, an individual can be a bank customer whether or not she has a loan with the bank.

Hence, it is possible that only some of the customer entities are related to the loan entity set through the borrower relationship, and the participation of customer in the borrower relationship set is therefore partial.



Keys

A key allows us to identify a set of attributes that suffice to distinguish entities from each other. Keys also help uniquely identify relationships, and thus distinguish relationships from each other.

a) Entity Sets

A **superkey** is a set of one or more attributes that, taken collectively, allow us to identify uniquely an entity in the entity set. For example, the customer-id attribute of the entity set customer is sufficient to distinguish one customer entity from another. Thus, customer-id is a superkey. Similarly, the combination of customer-name and customer-id is a superkey for the entity set customer. The customer-name attribute of customer is not a superkey, because several people might have the same name. The concept of a superkey is not sufficient for our purposes, since, as we saw, a superkey may contain extraneous attributes. If K is a superkey, then so is any superset of K . We are often interested in superkeys for which no proper subset is a superkey. Such minimal superkeys are called **candidate keys**. It is possible that several distinct sets of attributes could serve as a candidate key.

Although the attributes customerid and customer-name together can distinguish customer entities, their combination does not form a candidate key, since the attribute customer-id alone is a candidate key. We shall use the term **primary key** to denote a candidate key that is chosen by the database designer as the principal means of identifying entities within an entity set. A key (primary, candidate, and super) is a property of the entity set, rather than of the individual entities. Any two individual entities in the set are prohibited from having the same value on the key attributes at the same time.

The primary key should be chosen such that its attributes are never, or very rarely, changed. For instance, the address field of a person should not be part of the primary key, since it is likely to

change. Social-security numbers, on the other hand, are guaranteed to never change. Unique

identifiers generated by enterprises generally do not change, except if two enterprises merge; in such a case the same identifier may have been issued by both enterprises, and a reallocation of identifiers may be required to make sure they are unique.

b) Relationship sets

The primary key of an entity set allows us to distinguish among the various entities of the set. Let R be a relationship set involving entity sets $E_1, E_2, \dots, E_n$. Let $\text{primary-key}(E_i)$ denote the set of attributes that forms the primary key for entity set E_i . Assume for now that the attribute names of all primary keys are unique, and each entity set participates only once in the relationship. The composition of the primary key for a relationship set depends on the set of attributes associated with the relationship set R .

If the relationship set R has no attributes associated with it, then the set of attributes

$\text{primary-key}(E_1) \cup \text{primary-key}(E_2) \cup \dots \cup \text{primary-key}(E_n)$

describes an individual relationship in set R .

If the relationship set R has attributes $a_1, a_2, \dots, a_m$ associated with it, then the set of attributes

$\text{primary-key}(E_1) \cup \text{primary-key}(E_2) \cup \dots \cup \text{primary-key}(E_n) \cup \{a_1, a_2, \dots, a_m\}$

describes an individual relationship in set R .

In both of the above cases, the set of attributes

$\text{primary-key}(E_1) \cup \text{primary-key}(E_2) \cup \dots \cup \text{primary-key}(E_n)$

forms a superkey for the relationship set.

In case the attribute names of primary keys are not unique across entity sets, the attributes are renamed to distinguish them; the name of the entity set combined with the name of the attribute would form a unique name. The role name is used instead of the name of the entity set, to form a unique attribute name. The structure of the primary key for the relationship set depends on the mapping cardinality of the relationship set.

4. Explain in detail about design issues? (11 Marks)

a) Use of Entity Sets versus Attributes

Consider the entity set employee with attributes employee-name and telephone-number. We must redefine the employee entity set as:

- The employee entity set with attribute employee-name
- The telephone entity set with attributes telephone-number and location
- The relationship set emp-telephone, which denotes the association between employees

and the telephones that they have.

Treating a telephone as an attribute telephone-number implies that employees have precisely one telephone number each. Treating a telephone as an entity telephone permits employees to have several telephone numbers (including zero) associated with them. However, we could instead easily define telephone-number as a multivalued attribute to allow multiple telephones per employee. The main difference then is that treating a telephone as an entity better models a situation where one may want to keep extra information about a telephone, such as its location, or its type (mobile, video phone, or plain old telephone), or who all share the telephone. Thus, treating telephone as an entity is more general than treating it as an attribute and is appropriate when the generality may be useful.

b) Use of Entity Sets versus Relationship Sets

We assumed that a bank loan is modeled as an entity. An alternative is to model a loan not as an entity, but rather as a relationship between customers and branches, with loan-number and amount as descriptive attributes. Each loan is represented by a relationship between a customer and a branch. If every loan is held by exactly one customer and is associated with exactly one branch, we may find satisfactory the design where a loan is represented as a relationship. However, with this design, we cannot represent conveniently a situation in which several customers hold a loan jointly. To handle such a situation, we must define a separate relationship for each holder of the joint loan. Then, we must replicate the values for the descriptive attributes loan-number and amount in each such relationship. Each such relationship must, of course, have the same value for the descriptive attributes loan-number and amount. Two problems arise as a result of the replication: (1) the data are stored multiple times, wasting storage space, and (2) updates potentially leave the data in an inconsistent state, where the values differ in two relationships for attributes that are supposed to have the same value. The problem of replication of the attributes loan-number and amount is absent in the original design, because there loan is an entity set. One possible guideline in determining whether to use an entity set or a relationship set is to designate a relationship set to describe an action that occurs between entities. This approach can also be useful in deciding whether certain attributes may be more appropriately expressed as relationships.

Binary versus n-ary Relationship Sets

Relationships in databases are often binary. Some relationships that appear to be nonbinary could actually be better represented by several binary relationships. For instance, one could create a ternary relationship parent, relating a child to his/her mother and father. However, such a relationship could also be represented by two binary relationships, mother and father, relating a child to his/her mother and father separately. Using the two relationships mother and father

allows us record a child's mother, even if we are not aware of the father's identity; a null value would be required if the ternary relationship parent is used.

We replace the relationship set R by an entity set E, and create three relationship sets:

- RA, relating E and A
- RB, relating E and B
- RC, relating E and C

If the relationship set R had any attributes, these are assigned to entity set E; further, a special identifying attribute is created for E (since it must be possible to distinguish different entities in an entity set on the basis of their attribute values). For each relationship (a_i, b_i, c_i) in the relationship set R, we create a new entity e_i in the entity set E. Then, in each of the three new relationship sets, we insert a relationship as follows:

- (e_i, a_i) in RA
- (e_i, b_i) in RB
- (e_i, c_i) in RC

We can generalize this process in a straightforward manner to n-ary relationship sets. Thus, conceptually, we can restrict the E-R model to include only binary relationship sets. However, this restriction is not always desirable.

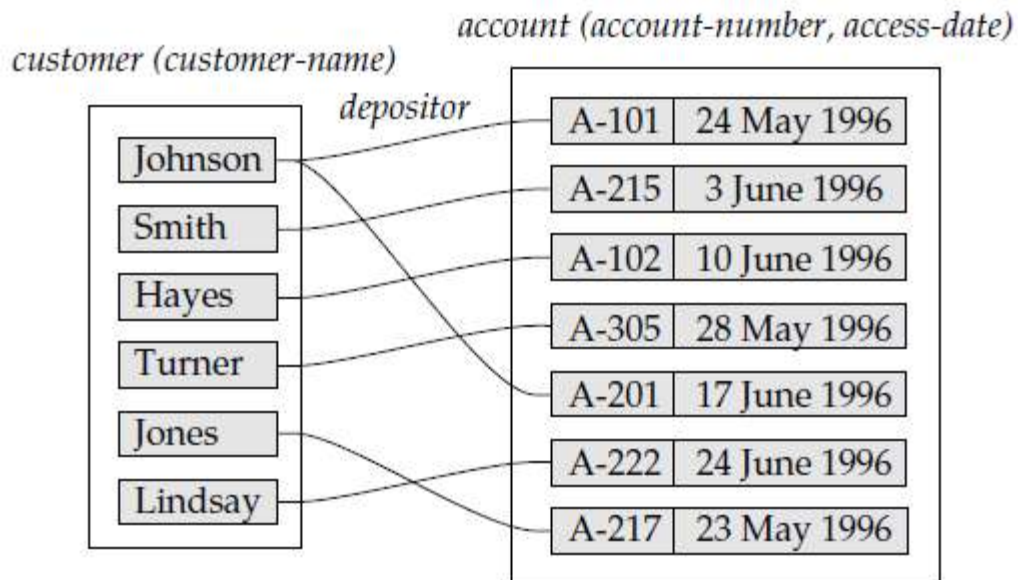
- An identifying attribute may have to be created for the entity set created to represent the relationship set. This attribute, along with the extra relationship sets required, increases the complexity of the design and overall storage requirements.
- A n-ary relationship set shows more clearly that several entities participate in a single relationship.
- There may not be a way to translate constraints on the ternary relationship into constraints on the binary relationships.

Placement of Relationship Attributes

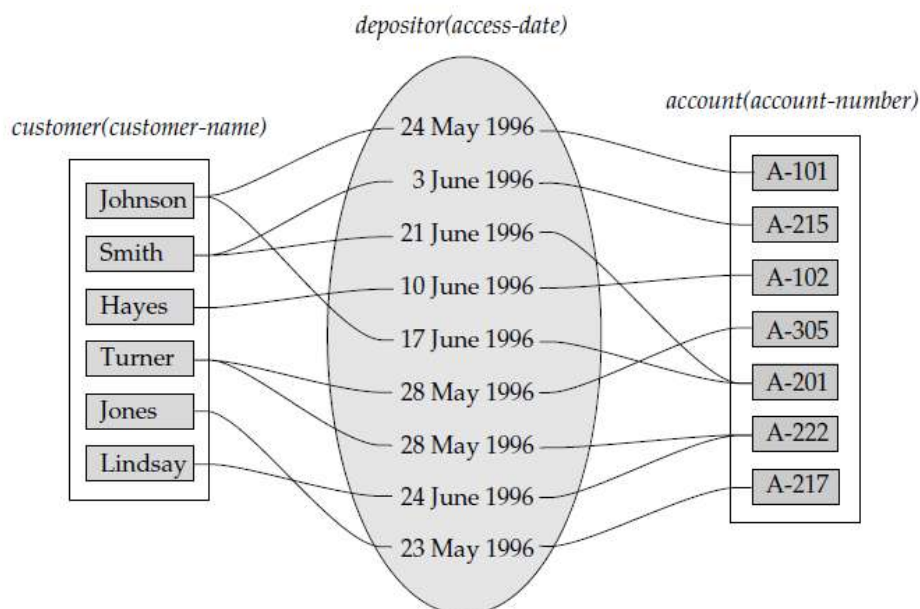
The cardinality ratio of a relationship can affect the placement of relationship attributes. Thus, attributes of one-to-one or one-to-many relationship sets can be associated with one of the participating entity sets, rather than with the relationship set. For instance, let us specify that depositor is a one-to-many relationship set such that one customer may have several accounts, but each account is held by only one customer. In this case, the attribute access-date, which specifies when the customer last accessed that account, could be associated with the account entity set.

The choice of attribute placement is more clear-cut for many-to-many relationship sets. Example, let us specify the perhaps more realistic case that depositor is a many-to-many relationship set expressing that a customer may have one or more accounts, and that an account can be held by

one or more customers. If we are to express the date on which a specific customer last accessed a specific account, access-date must be an attribute of the depositor relationship set, rather than either one of the participating entities. If access-date were an attribute of account, for instance, we could not determine which customer made the most recent access to a joint account. When an attribute is determined by the combination of participating entity sets, rather than by either entity separately, that attribute must be associated with the many-to-many relationship set.



Access-date as attribute of the account entity set.



Access-date as attribute of the depositor relationship set.

5. Explain Entity-Relationship Diagram? (11 Marks) **APRIL 2014, NOV 2015(DEC2018)(MAY 2019)**

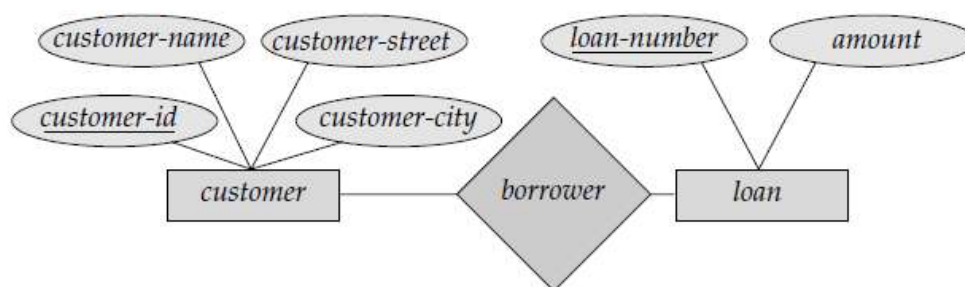
The following major components:

- **Rectangles**, which represent entity sets
- **Ellipses**, which represent attributes
- **Diamonds**, which represent relationship sets
- **Lines**, which link attributes to entity sets and entity sets to relationship sets
- **Double ellipses**, which represent multivalued attributes
- **Dashed ellipses**, which denote derived attributes
- **Double lines**, which indicate total participation of an entity in a relationship set
- **Double rectangles**, which represent weak entity

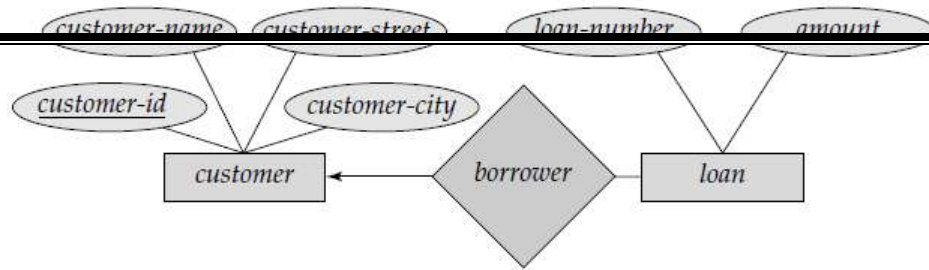
Consider the entity-relationship diagram which consists of two entity sets, customer and loan, related through a binary relationship set borrower. The attributes associated with customer are customer-id, customer-name, customer-street, and customer-city. The attributes associated with loan are loan-number and amount.

The relationship set borrower may be many-to-many, one-to-many, many-to-one, or one-to-one. To distinguish among these types, we draw either a directed line ($\rightarrow$) or an undirected line ($—$) between the relationship set and the entity set in question.

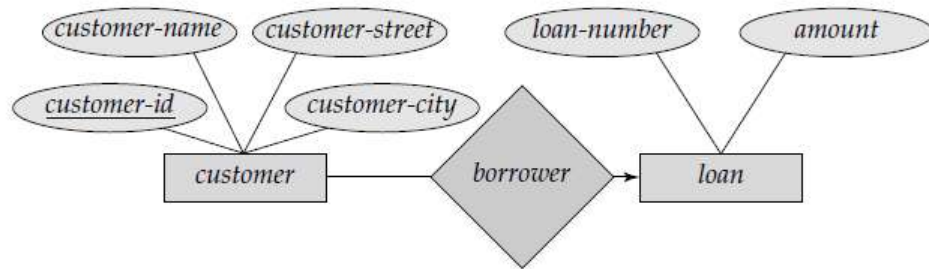
- A directed line from the relationship set borrower to the entity set loan specifies that borrower is either a one-to-one or many-to-one relationship set, from customer to loan; borrower cannot be a many-to-many or a one-to-many relationship set from customer to loan.
- An undirected line from the relationship set borrower to the entity set loan specifies that borrower is either a many-to-many or one-to-many relationship set from customer to loan.



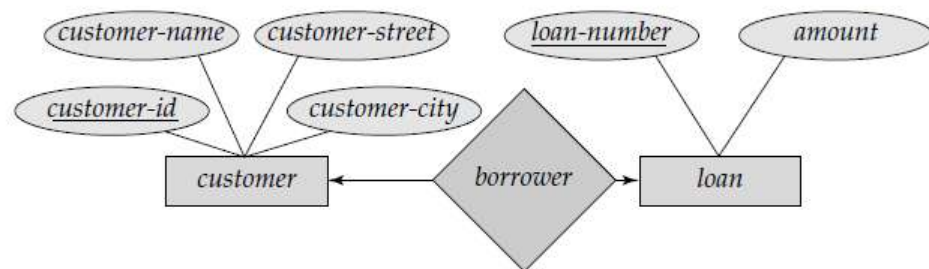
E-R diagram corresponding to customers and loans.



(a)

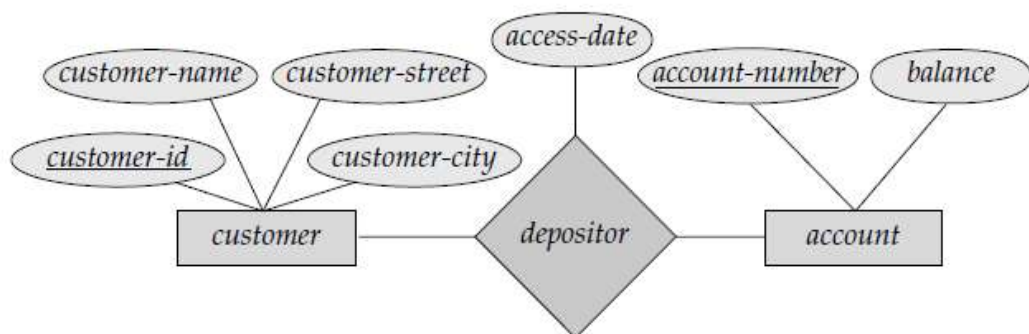


(b)



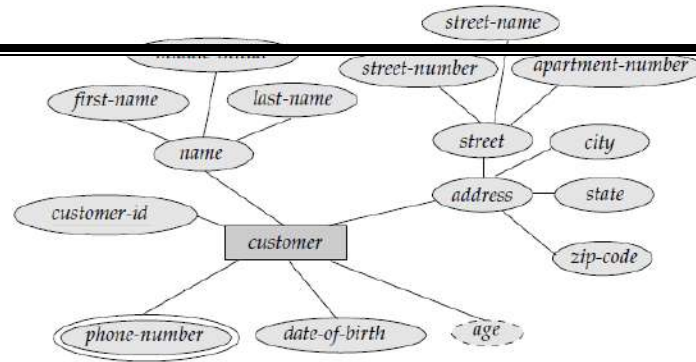
(c)

Relationships. (a) one to many. (b) many to one. (c) one-to-one.

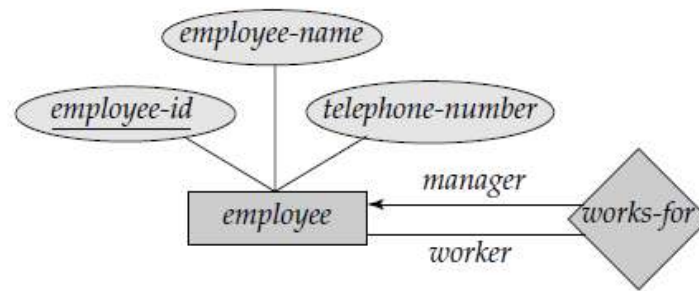


E-R diagram with an attribute attached to a relationship set.

A multivalued attribute phone-number, depicted by a double ellipse, and a derived attribute age, depicted by a dashed ellipse.



E-R diagram with composite, multivalued, and derived attributes.



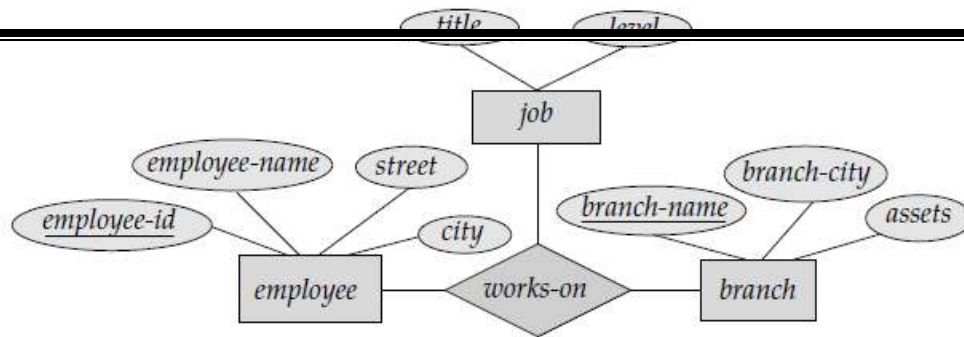
E-R diagram with role indicators

The role indicators manager and worker between the employee entity set and the works-for relationship set. Nonbinary relationship sets can be specified easily in an E-R diagram.

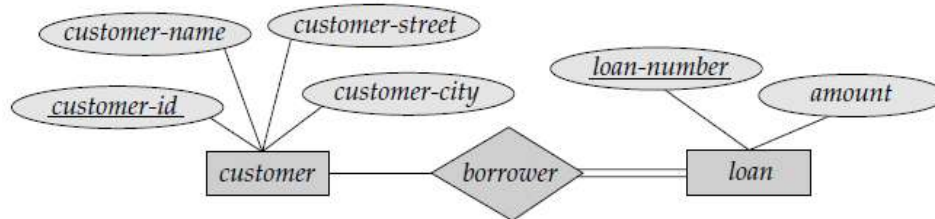
This consists of the three entity sets employee, job, and branch, related through the relationship set works-on.

Suppose there is a relationship set R between entity sets $A_1, A_2, \dots, A_n$, and the only arrows are on the edges to entity sets $A_{i+1}, A_{i+2}, \dots, A_n$. Then, the two possible interpretations are:

1. A particular combination of entities from $A_1, A_2, \dots, A_i$ can be associated with at most one combination of entities from $A_{i+1}, A_{i+2}, \dots, A_n$. Thus, the primarykey for the relationship R can be constructed by the union of the primary keys of $A_1, A_2, \dots, A_i$.
2. For each entity set A_k , $i < k \leq n$, each combination of the entities from the other entity sets can be associated with at most one entity from A_k . Each set $\{A_1, A_2, \dots, A_{k-1}, A_{k+1}, \dots, A_n\}$, for $i < k \leq n$, then forms a candidate key.



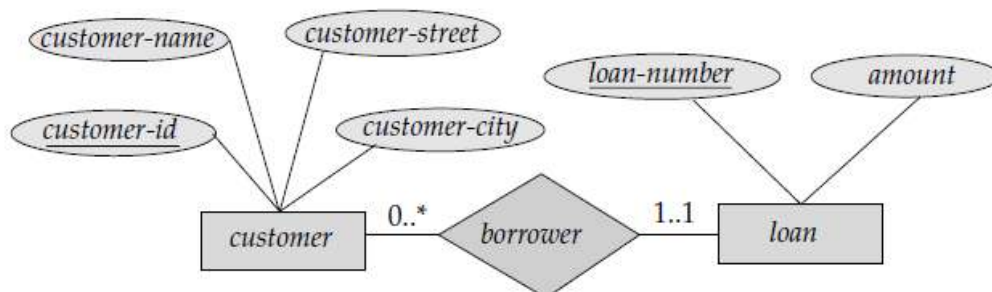
E-R diagram with a ternary relationship.



Total participation of an entity set in a relationship set.

Double lines are used in an E-R diagram to indicate that the participation of an entity set in a relationship set is total; that is, each entity in the entity set occurs in at least one relationship in that relationship set. For instance, consider the relationship borrower between customers and loans. A double line from loan to borrower, as in Figure 2.14, indicates that each loan must have at least one associated customer.

It is easy to misinterpret the 0..* on the edge between customer and borrower, and think that the relationship borrower is many to one from customer to loan—this is exactly the reverse of the correct interpretation. If both edges from a binary relationship have a maximum value of 1, the relationship is one to one. If we had specified a cardinality limit of 1..* on the edge between customer and borrower, we would be saying that each customer must have at least one loan.



Cardinality limits on relationship sets.

6. Explain Weak Entity Sets? (11 Marks)

An entity set may not have sufficient attributes to form a primary key. Such an entity set is termed a **weak entity set**. An entity set that has a primary key is termed a **strong entity set**.

Consider the entity set payment, which has the three attributes: payment-number, payment-date, and payment-amount. Payment numbers are typically sequential numbers, starting from 1, generated separately for each loan. Thus, although each payment entity is distinct, payments for different loans may share the same payment number. Thus, this entity set does not have a primary key; it is a weak entity set.

For a weak entity set to be meaningful, it must be associated with another entity set, called the **identifying or owner entity set**. Every weak entity must be associated with an identifying entity; that is, the weak entity set is said to be **existence dependent** on the identifying entity set. The identifying entity set is said to **own** the weak entity set that it identifies. The relationship associating the weak entity set with the identifying entity set is called the **identifying relationship**. The identifying relationship is many to one from the weak entity set to the identifying entity set, and the participation of the weak entity set in the relationship is total.

Although a weak entity set does not have a primary key, we nevertheless need a means of distinguishing among all those entities in the weak entity set that depend on one particular strong entity. The **discriminator** of a weak entity set is a set of attributes that allows this distinction to be made.

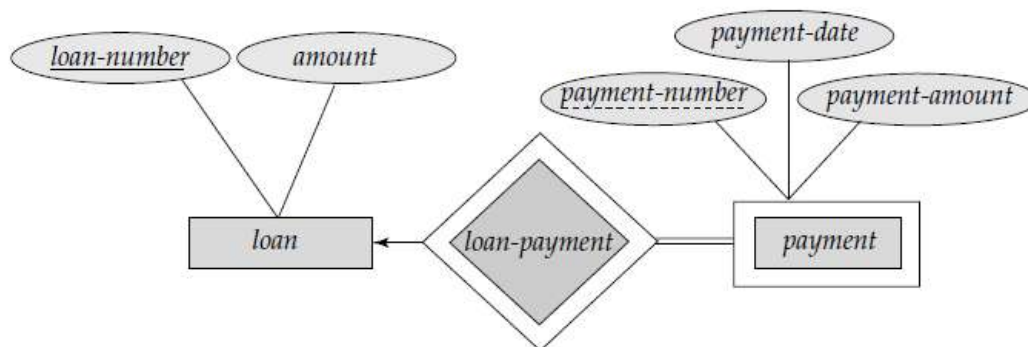
For example, the discriminator of the weak entity set payment is the attribute payment-number, since, for each loan, a payment number uniquely identifies one single payment for that loan. The discriminator of a weak entity set is also called the partial key of the entity set. The primary key of a weak entity set is formed by the primary key of the identifying entity set, plus the weak entity set's discriminator.

The identifying relationship set should have no descriptive attributes, since any required attributes can be associated with the weak entity set.

A weak entity set can participate in relationships other than the identifying relationship. For instance, the payment entity could participate in a relationship with the account entity set, identifying the account from which the payment was made. A weak entity set may participate as owner in an identifying relationship with another weak entity set. It is also possible to have a weak entity set with more than one identifying entity set. A particular weak entity would then be identified by a combination of entities, one from each identifying entity set. The primary key of the weak entity set would consist of the union of the primary keys of the identifying entity sets, plus the discriminator of the weak entity set.

In E-R diagrams, a doubly outlined box indicates a weak entity set, and a doubly outlined diamond indicates the corresponding identifying relationship.

The database designer may choose to express a weak entity set as a multivalued composite attribute of the owner entity set. In our example, this alternative would require that the entity set loan have a multivalued, composite attribute payment, consisting of payment-number, payment-date, and payment amount. A weak entity set may be more appropriately modeled as an attribute if it participates in only the identifying relationship, and if it has few attributes.



E-R diagram with a weak entity set.

7. Explain specialization and generalization? (11 Marks) [DEC 2016]

Extended E-R Features [DEC 2016]

Specialization

An entity set may include subgroupings of entities that are distinct in some way from other entities in the set. For instance, a subset of entities within an entity set may have attributes that are not shared by all the entities in the entity set. The E-R model provides a means for representing these distinctive entity groupings. Consider an entity set person, with attributes name, street, and city. A person may be further classified as one of the following:

- customer
- employee

Each of these person types is described by a set of attributes that includes all the attributes of entity set person plus possibly additional attributes. For example, customer entities may be described further by the attribute customer-id, whereas employee entities may be described further by the attributes employee-id and salary. The process of designating subgroupings within an entity set is called **specialization**. The specialization of person allows us to distinguish among persons according to whether they are employees or customers.

The bank could then create two specializations of account, namely savings-account and checking account. The entity set savings-account would have all the attributes of account and an additional attribute interest-rate. The entity set checking account would have all the attributes of account, and an additional attribute overdraft amount.

We can apply specialization repeatedly to refine a design scheme. For instance, bank employees may be further classified as one of the following:

- officer
- teller
- secretary

Each of these employee types is described by a set of attributes that includes all the attributes of entity set employee plus additional attributes. For example, officer entities may be described further by the attribute office-number, teller entities by the attributes station-number and hours-per-week, and secretary entities by the attribute hours-per week. Further, secretary entities may participate in a relationship secretary-for, which identifies which employees are assisted by a secretary. An entity set may be specialized by more than one distinguishing feature.

In terms of an E-R diagram, specialization is depicted by a triangle component labeled **ISA**. The label ISA stands for “is a” and represents, for example, that a customer “is a” person. The ISA relationship may also be referred to as a **superclass-subclass** relationship. Higher- and lower-level entity sets are depicted as regular entity set that is, as rectangles containing the name of the entity set.

Generalization

The refinement from an initial entity set into successive levels of entity subgroupings represents a **top down** design process in which distinctions are made explicit. The design process may also proceed in a **bottom-up** manner, in which multiple entity sets are synthesized into a higher-level entity set on the basis of common features. The database designer may have first identified a customer entity set with the attributes name, street, city, and customer-id, and an employee entity set with the attributes name, street, city, employee-id, and salary.

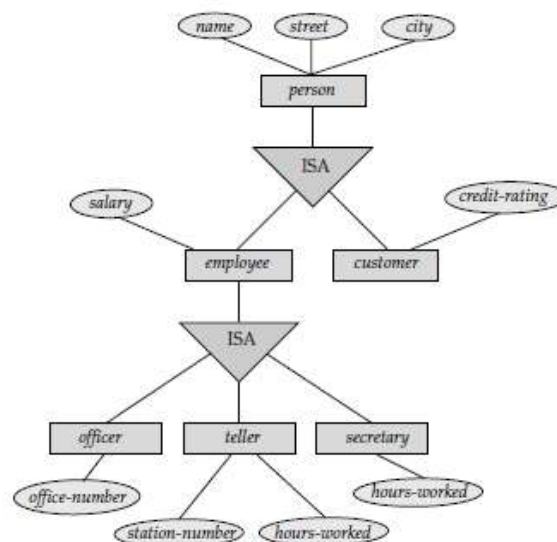
There are similarities between the customer entity set and the employee entity set in the sense that they have several attributes in common. This commonality can be expressed by **generalization**, which is a containment relationship that exists between a higher-level entity set and one or more lower-level entity sets. In our example, person is the higher-level entity set and customer and employee are lower-level entity sets.

Higher- and lower-level entity sets also may be designated by the terms **superclass** and **subclass**, respectively. The person entity set is the superclass of the customer and employee subclasses. Generalization is a simple inversion of specialization.

Specialization stems from a single entity set; it emphasizes differences among entities within the set by creating distinct lower-level entity sets. These lower-level entity sets may have attributes, or may participate in relationships, that do not apply to all the entities in the higher-level entity set.

If customer and employee neither have attributes that person entities do not have nor participate in different relationships than those in which person entities participate, there would be no need to specialize the person entity set.

Generalization proceeds from the recognition that a number of entity sets share some common features (namely, they are described by the same attributes and participate in the same relationship sets). On the basis of their commonalities, generalization synthesizes these entity sets into a single, higher-level entity set. Generalization is used to emphasize the similarities among lower-level entity sets and to hide the differences; it also permits an economy of representation in that shared attributes are not repeated.



Specialization and generalization.

Attribute Inheritance

A crucial property of the higher- and lower-level entities created by specialization and generalization is **attribute inheritance**. The attributes of the higher-level entity sets are said to be **inherited** by the lower level entity sets. For example, customer and employee inherit the

attributes of person. Thus, customer is described by its name, street, and city attributes, and additionally a customer-id attribute; employee is described by its name, street, and city attributes, and additionally employee-id and salary attributes.

A lower-level entity set (or subclass) also inherits participation in the relationship sets in which its higher level entity (or superclass) participates. The officer, teller, and secretary entity sets can participate in the works-for relationship set, since the superclass employee participates in the works-for relationship. Attribute inheritance applies through all tiers of lower-level entity sets. The above entity sets can participate in any relationships in which the person entity set participates. Whether a given portion of an E-R model was arrived at by specialization or generalization, the outcome is basically the same:

- A higher-level entity set with attributes and relationships that apply to all of its lower-level entity sets
- Lower-level entity sets with distinctive features that apply only within a particular lower-level entity set

Employee is a lower-level entity set of person and a higher-level entity set of the officer, teller, and secretary entity sets. In a hierarchy, a given entity set may be involved as a lower-level entity set in only one ISA relationship; that is, entity sets in this diagram have only **single inheritance**. If an entity set is a lower-level entity set in more than one ISA relationship, then the entity set has **multiple inheritance**, and the resulting structure is said to be a lattice.

Constraints on Generalizations

the database designer may choose to place certain constraints on a particular generalization. One type of constraint involves determining which entities can be members of a given lower-level entity set. Such membership may be one of the following:

- **Condition-defined.** In condition-defined lower-level entity sets, membership is evaluated on the basis of whether or not an entity satisfies an explicit condition or predicate. For example, assume that the higher level entity set account has the attribute account-type. All account entities are evaluated on the defining account-type attribute. Only those entities that satisfy the condition account-type = "savings account" are allowed to belong to the lower-level entity set person. All entities that satisfy the condition account-type = "checking account" are included in checking account. Since all the lower-level entities are evaluated on the basis of the same attribute (in this case, on account-type), this type of generalization is said to be **attribute-defined**.

- **User-defined.** User-defined lower-level entity sets are not constrained by a membership condition; rather, the database user assigns entities to a given entity set.

A second type of constraint relates to whether or not entities may belong to more than one lower-level entity set within a single generalization. The lower-level entity sets may be one of the following:

- **Disjoint.** A disjointness constraint requires that an entity belong to no more than one lower-level entity set. In our example, an account entity can satisfy only one condition for the account-type attribute; an entity can be either a savings account or a checking account, but cannot be both.
- **Overlapping.** In overlapping generalizations, the same entity may belong to more than one lower-level entity set within a single generalization. A given employee may therefore appear in more than one of the team entity sets that are lower-level entity sets of employee. Thus, the generalization is overlapping. As another example, suppose generalization applied to entity sets customer and employee leads to a higher level entity set person. The generalization is overlapping if an employee can also be a customer.

A final constraint, the **completeness constraint** on a generalization or specialization, specifies whether or not an entity in the higher-level entity set must belong to at least one of the lower-level entity sets within the generalization/specialization. This constraint may be one of the following:

- **Total generalization or specialization.** Each higher-level entity must belong to a lower-level entity set.
- **Partial generalization or specialization.** Some higher-level entities may not belong to any lower-level entity set.

Partial generalization is the default. We can specify total generalization in an E-R diagram by using a double line to connect the box representing the higher-level entity set to the triangle symbol.

8. Explain Aggregation? (11 Marks)

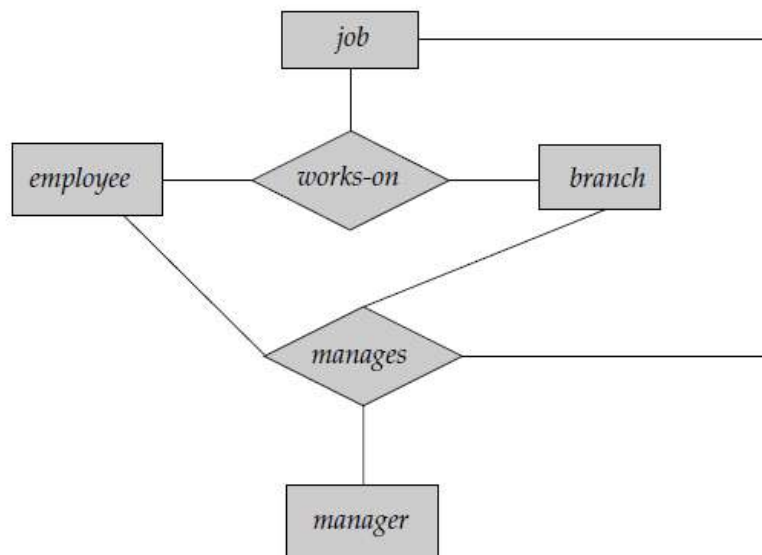
One limitation of the E-R model is that it cannot express relationships among relationships

One alternative for representing this relationship is to create a quaternary relationship manages between employee, branch, job, and manager. (A quaternary relationship is required—a binary

relationship between manager and employee would not permit us to represent which (branch, job) combinations or an employee are managed by which manager.)

It appears that the relationship sets *works-on* and *manages* can be combined into one single relationship set. If the manager were a value rather than an *manager* entity, we could instead make *manager* a multivalued attribute of the relationship *works-on*.

Aggregation is an abstraction through which relationships are treated as higher level entities.

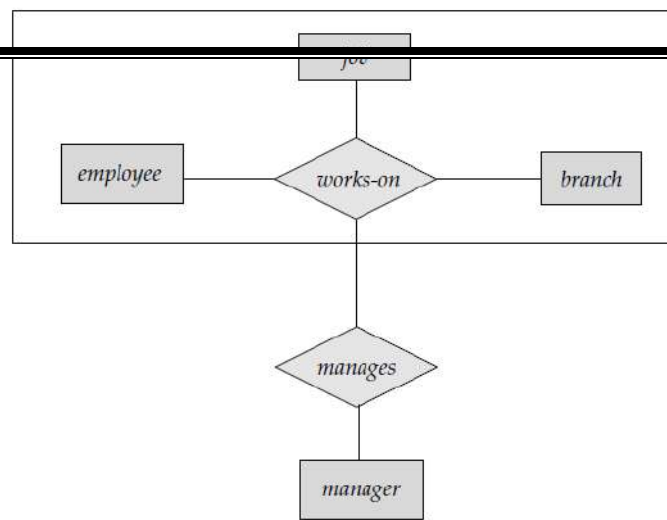


E-R diagram with redundant relationships.

Alternative E-R Notations

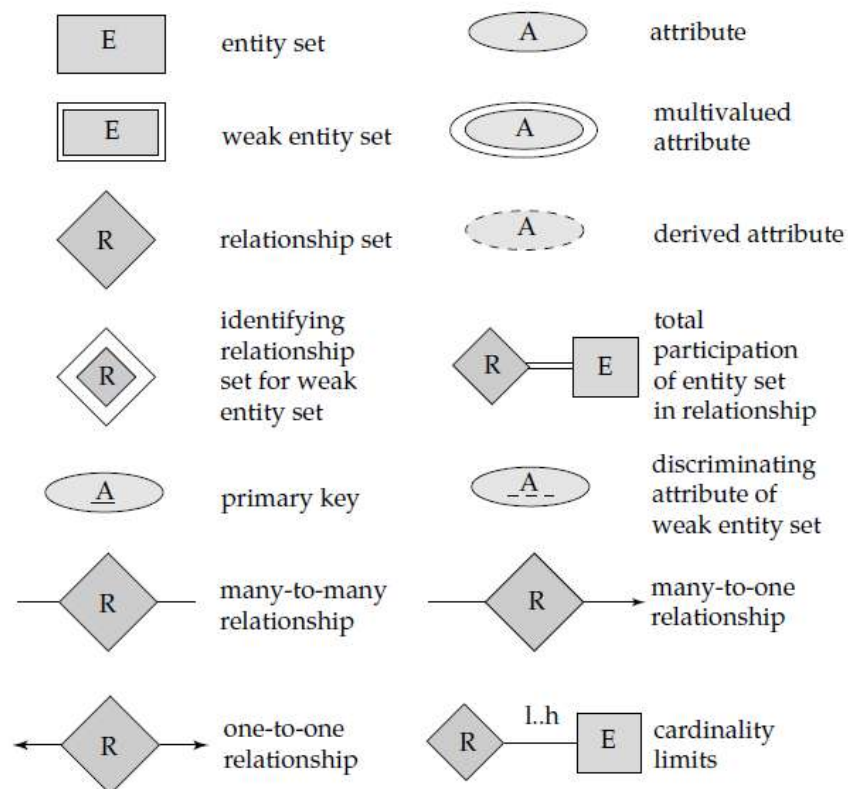
An entity set may be represented as a box with the name outside, and the attributes listed one below the other within the box. The primary key attributes are indicated by listing them at the top, with a line separating them from the other attributes.

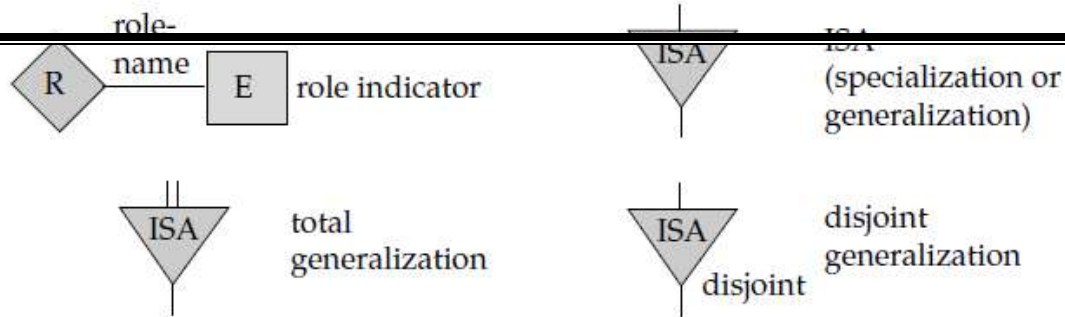
Cardinality constraints can be indicated in several different ways. The labels * and 1 on the edges out of the relationship are sometimes used for depicting many-to-many, one-to-one, and many-to-one relationships. Cardinality constraints in such a notation are shown by “crow’s foot” notation.



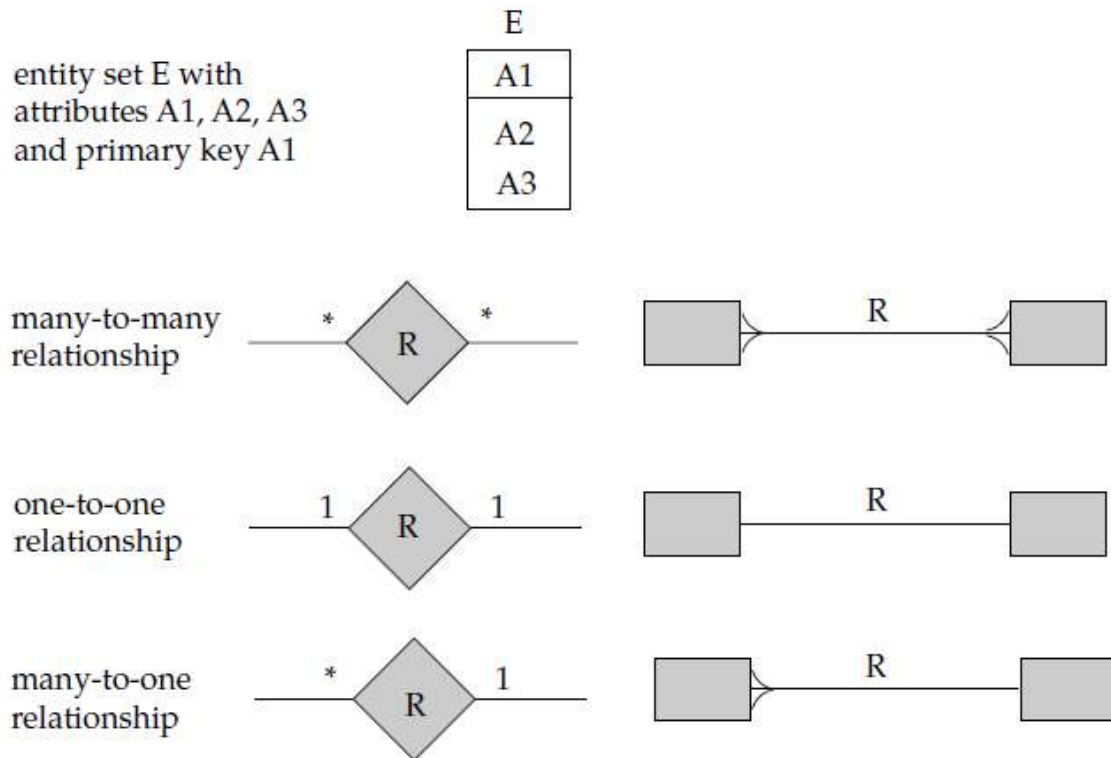
E-R diagram with aggregation

Design of an E-R Database Schema





Symbols used in the E-R notation.



Alternative E-R notations.

9. Explain about Design Phases? (11 Marks)NOV 2014

A high-level data model serves the database designer by providing a conceptual framework in which to specify, in a systematic fashion, what the data requirements of the database users are, and how the database will be structured to fulfill these requirements. The initial phase of database design, then, is to characterize fully the data needs of the prospective database users. The database designer needs to interact extensively with domain experts and users to carry out this task. The outcome of this phase is a specification of user requirements.

Next, the designer chooses a data model, and by applying the concepts of the chosen data model, translates these requirements into a conceptual schema of the database. The schema developed at this **conceptual-design** phase provides a detailed overview of the enterprise.

In a **specification of functional requirements**, users describe the kinds of operations (or transactions) that will be performed on the data. Example operations include modifying or updating data, searching for and retrieving specific data, and deleting data.

Database Design for Banking Enterprise

Data Requirements

The major characteristics of the banking enterprise:

- The bank is organized into branches. Each branch is located in a particular city and is identified by a unique name. The bank monitors the assets of each branch.
- Bank customers are identified by their customer-id values. The bank stores each customer's name, and the street and city where the customer lives. Customer may have accounts and can take out loans. A customer may be associated with a particular banker, who may act as a loan officer or personal banker for that customer.
- Bank employees are identified by their employee-id values. The bank administration stores the name and telephone number of each employee, the names of the employee's dependents, and the employee-id number of the employee's manager. The bank also keeps track of the employee's start date and, thus, length of employment.
- The bank offers two types of accounts savings and checking accounts. Accounts can be held by more than one customer, and a customer can have more than one account. Each account is assigned a unique account number. The bank maintains a record of each account's balance, and the most recent date on which the account was accessed by each customer holding the account. In addition, each savings account has an interest rate, and overdrafts are recorded for each checking account.
- A loan originates at a particular branch and can be held by one or more customers. A loan is identified by a unique loan number. For each loan, the bank keeps track of the loan amount and the loan payments. Although a loanpayment number does not uniquely identify a particular payment among those for all the bank's loans, a payment number does identify a particular payment for a specific loan. The date and amount are recorded for each payment.

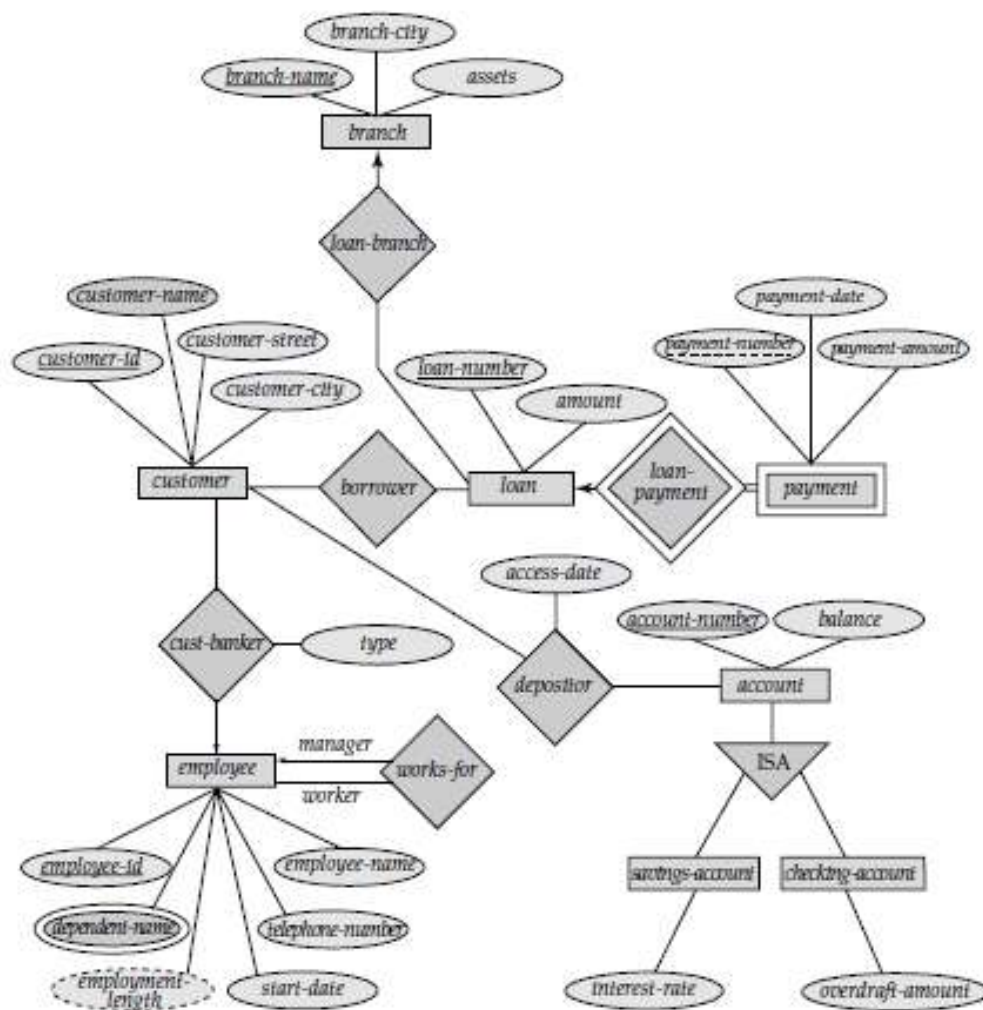
Entity Sets Designation

- The branch entity set, with attributes branch-name, branch-city, and assets.
- The customer entity set, with attributes customer-id, customer-name, customerstreet; and customer-city. A possible additional attribute is banker-name.
- The employee entity set, with attributes employee-id, employee-name, telephonenumber, salary, and manager. Additional descriptive features are the multivalued attribute dependent-name, the base attribute start-date, and the derived attribute employment-length.
- Two account entity sets—savings-account and checking-account—with the common attributes of account-number and balance; in addition, savings-account has the attribute interest-rate and checking account has the attribute overdraft-amount.
- The loan entity set, with the attributes loan-number, amount, and originatingbranch.
- The weak entity set loan-payment, with attributes payment-number, paymentdate, and payment-amount.

Relationship Sets Designation

- borrower, a many-to-many relationship set between customer and loan.
- loan-branch, a many-to-one relationship set that indicates in which branch a loan originated.
- loan-payment, a one-to-many relationship from loan to payment, which documents that a payment is made on a loan.
- depositor, with relationship attribute access-date, a many-to-many relationship set between customer and account, indicating that a customer owns an account.
- cust-banker, with relationship attribute type, a many-to-one relationship set expressing that a customer can be advised by a bank employee, and that a bank employee can advise one or more customers. Note that this relationship set has replaced the attribute banker-name of the entity set customer.
- works-for, a relationship set between employee entities with role indicators manager and worker; the mapping cardinalities express that an employee worksfor only one manager and that a manager supervises one or more employees.

E-R Diagram



Reduction of an E-R Schema to Tables

Tabular Representation of Strong Entity Sets

Let E be a strong entity set with descriptive attributes $a_1, a_2, \dots, a_n$. We represent this entity by a table called E with n distinct columns, each of which corresponds to one of the attributes of E . Each row in this table corresponds to one entity of the entity set E .

Consider the entity set loan of the E-R diagram. This entity set has two attributes: loan-number and amount . We represent this entity set by a table called loan , with two columns.

Let D_1 denote the set of all loan numbers, and let D_2 denote the set of all balances. Any row of the loan table must consist of a 2-tuple (v_1, v_2) , where v_1 is a loan (that is, v_1 is in set D_1) and v_2 is an amount (that is, v_2 is in set D_2). In general, the loan table will contain only a subset of the set of all possible rows. We refer to the set of all possible rows of loan as the Cartesian product of D_1 and D_2 , denoted by $D_1 \times D_2$.

In general, if we have a table of n columns, we denote the Cartesian product of $D_1, D_2, \dots, D_n$ by

<i>loan-number</i>	<i>amount</i>
L-11	900
L-14	1500
L-15	1500
L-16	1300
L-17	1000
L-23	2000
L-93	500

<i>customer-id</i>	<i>customer-name</i>	<i>customer-street</i>	<i>customer-city</i>
019-28-3746	Smith	North	Rye
182-73-6091	Turner	Putnam	Stamford
192-83-7465	Johnson	Alma	Palo Alto
244-66-8800	Curry	North	Rye
321-12-3123	Jones	Main	Harrison
335-57-7991	Adams	Spring	Pittsfield
336-66-9999	Lindsay	Park	Pittsfield
677-89-9011	Hayes	Main	Harrison
963-96-3963	Williams	Nassau	Princeton

Tabular Representation of Weak Entity Sets

Let A be a weak entity set with attributes $a_1, a_2, \dots, a_m$. Let B be the strong entity set on which A depends. Let the primary key of B consist of attributes $b_1, b_2, \dots, b_n$. We represent the entity set A by a table called A with one column for each attribute of the set:

$$\{a_1, a_2, \dots, a_m\} \cup \{b_1, b_2, \dots, b_n\}$$

This entity set has three attributes: payment-number, payment-date, and payment-amount. The primary key of the loan entity set, on which payment depends, is loan-number. Thus, we represent payment by a table with four columns labeled loan-number, paymentnumber, payment-date, and payment-amount.

Tabular Representation of Relationship Sets

Let R be a relationship set, let $a_1, a_2, \dots, a_m$ be the set of attributes formed by the union of the primary keys of each of the entity sets participating in R , and let the descriptive attributes (if any) of R be $b_1, b_2, \dots, b_n$. We represent this relationship set by a table called R with one column for each attribute of the set:

$$\{a_1, a_2, \dots, a_m\} \cup \{b_1, b_2, \dots, b_n\}$$

This relationship set involves the following two entity sets:

- customer, with the primary key customer-id
- loan, with the primary key loan-number

Since the relationship set has no attributes, the borrower table has two columns, labelled *customer-id* and *loan-number*.

<i>loan-number</i>	<i>payment-number</i>	<i>payment-date</i>	<i>payment-amount</i>
L-11	53	7 June 2001	125
L-14	69	28 May 2001	500
L-15	22	23 May 2001	300
L-16	58	18 June 2001	135
L-17	5	10 May 2001	50
L-17	6	7 June 2001	50
L-17	7	17 June 2001	100
L-23	11	17 May 2001	75
L-93	103	3 June 2001	900
L-93	104	13 June 2001	200

Redundancy of Tables

A relationship set linking a weak entity set to the corresponding strong entity set is treated specially. The weak entity set payment is dependent on the strong entity set loan via the relationship set loan payment. The primary key of payment is {loan-number, payment-number}, and the primary key of loan is {loan-number}. Since loan-payment has no descriptive attributes, the loan-payment table would have two columns, loan-number and payment number. The table for the entity set payment has four columns, loan-number, payment number, payment-date, and payment-amount. Every(loan-number, payment-number) combination in loan-payment would also be present in the payment table, and vice versa. Thus, the loan-payment table is redundant.

<i>customer-id</i>	<i>loan-number</i>
019-28-3746	L-11
019-28-3746	L-23
244-66-8800	L-93
321-12-3123	L-17
335-57-7991	L-16
555-55-5555	L-14
677-89-9011	L-15
963-96-3963	L-17

Combination of Tables

Consider a many-to-one relationship set AB from entity set A to entity set B. Using our table construction scheme outlined previously, we get three tables: A, B, and AB. Suppose further that the participation of A in the relationship is total; that is, every entity a in the entity set A must participate in the relationship AB. Then we can combine the tables A and AB to form a single table consisting of the union of columns of both tables.

The double line in the E-R diagram indicates that the participation of account in the account-branch is total. Hence, an account cannot exist without being associated with a particular branch. Further, the relationship set account-branch is many to one from account to branch. Therefore, we can combine the table for account-branch with the table for account and require only the following two tables:

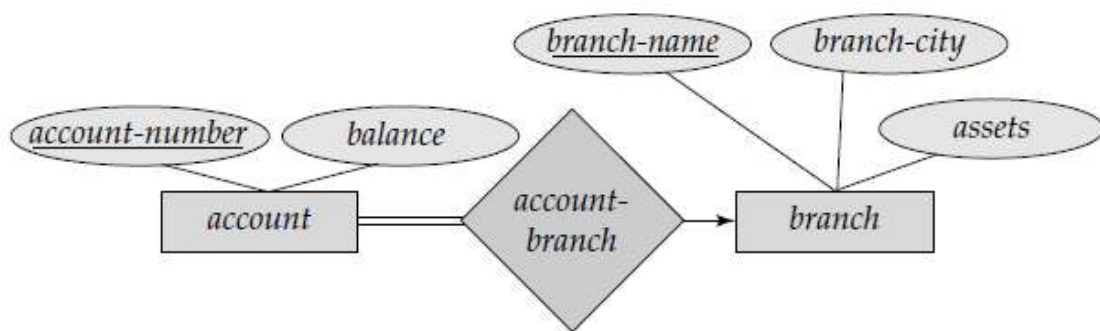
- account, with attributes account-number, balance, and branch-name
- branch, with attributes branch-name, branch-city, and assets

Composite Attributes (2 MARKS NOV 2010)

We handle composite attributes by creating a separate attribute for each of the component attributes; we do not create a separate column for the composite attribute itself. Suppose address is a composite attribute of entity set customer, and the components of address are street and city. The table generated from customer would then contain columns address-street and address-city; there is no separate column for address.

Multivalued Attributes

We have seen that attributes in an E-R diagram generally map directly into columns for the appropriate tables. Multivalued attributes, however, are an exception; new tables are created for these attributes.



Tabular Representation of Generalization

There are two different methods for transforming to a tabular form an E-R diagram that includes generalization.

1. Create a table for the higher-level entity set. For each lower-level entity set, create a table that includes a column for each of the attributes of that entity set plus a column for each attribute of the primary key of the higher-level entity set. We have three tables:

- account, with attributes account-number and balance

- savings-account, with attributes account-number and interest-rate

- checking-account, with attributes account-number and overdraft-amount

2. An alternative representation is possible, if the generalization is disjoint and complete—that is, if no entity is a member of two lower-level entity sets directly below a higher-level entity set, and if every entity in the higher level entity set is also a member of one of the lower-level entity sets. Here, do not create a table for the higher-level entity set. Instead, for each lower-level entity set, create a table that includes a column for each of the attributes of that entity set plus a column for each attribute of the higher-level entity set. We have two tables.

- savings-account, with attributes account-number, balance, and interest-rate
- checking-account, with attributes account-number, balance, and overdraftamount

The savings-account and checking-account relations corresponding to these tables both have account-number as the primary key.

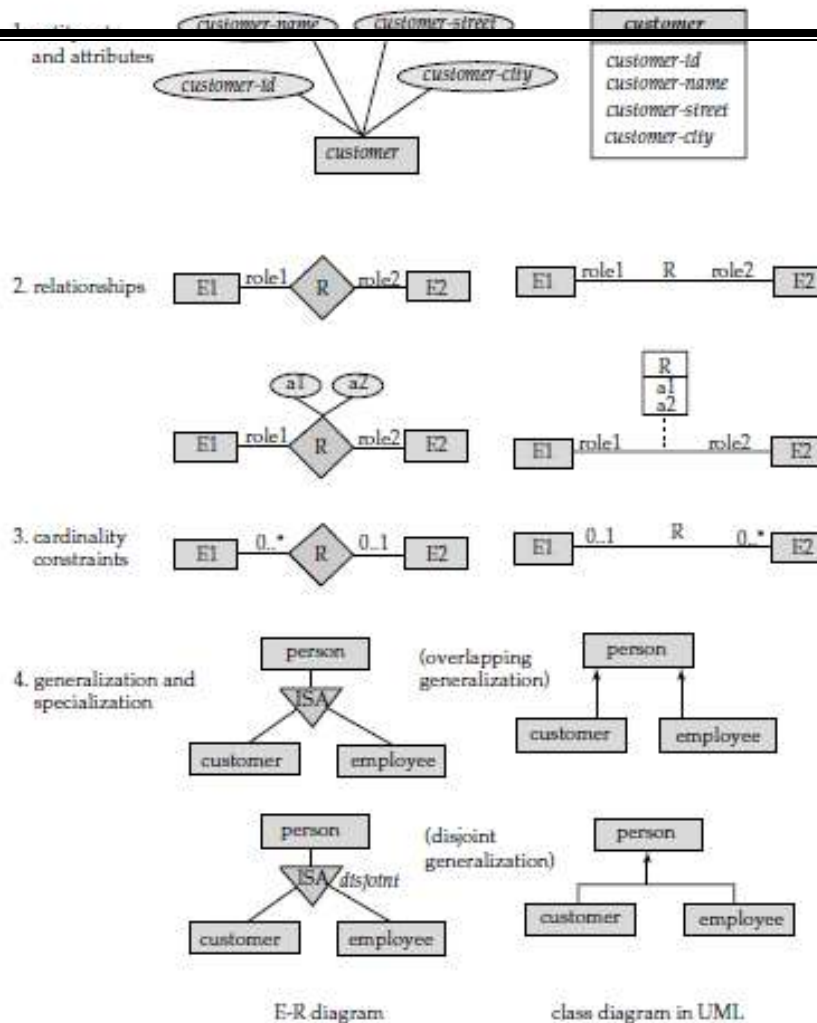
Tabular Representation of Aggregation

The table for the relationship set manages between the aggregation of works-on and the entity set manager includes a column for each attribute in the primary keys of the entity set manager and the relationship set works-on. It would also include a column for any descriptive attributes, if they exist, of the relationship set manages. We then transform the relationship sets and entity sets within the aggregated entity.

The Unified Modeling Language UML**

Entity-relationship diagrams help model the data representation component of a software system. Data representation, however, forms only one part of an overall system design. Other components include models of user interactions with the system, specification of functional modules of the system and their interaction, etc. The **Unified Modeling Language (UML)**, is a proposed standard for creating specifications of various components of a software system. Some of the parts of UML are:

- **Class diagram.** A class diagram is similar to an E-R diagram.
- **Use case diagram.** Use case diagrams show the interaction between users and the system, in particular the steps of tasks that users perform (such as withdrawing money or registering for a course).
- **Activity diagram.** Activity diagrams depict the flow of tasks between various components of a system.
- **Implementation diagram.** Implementation diagrams show the system components and their interconnections, both at the software component level and the hardware component level.



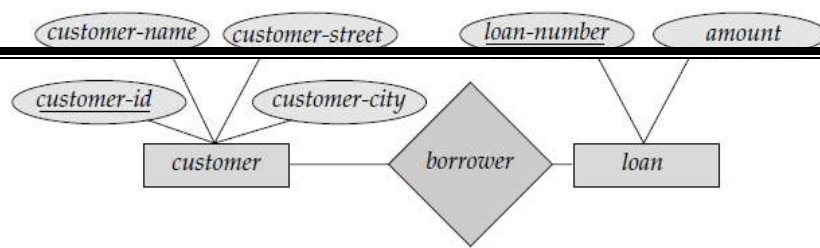
Symbols used in the UML class diagram notation.

Nonbinary relationships cannot be directly represented in UML. Cardinality constraints are specified in UML in the same way as in E-R diagrams, in the form l..h, where l denotes the minimum and h the maximum number of relationships an entity can participate in. However, you should be aware that the positioning of the constraints is exactly the reverse of the positioning of constraints in E-R diagrams,

16. Discuss the various concepts of ER Model with an example. **APRIL 2015**

The following major components:

- **Rectangles**, which represent entity sets
- **Ellipses**, which represent attributes
- **Diamonds**, which represent relationship sets
- **Lines**, which link attributes to entity sets and entity sets to relationship sets
- **Double ellipses**, which represent multivalued attributes
- **Dashed ellipses**, which denote derived attributes
- **Double lines**, which indicate total participation of an entity in a relationship set
- **Double rectangles**, which represent weak entity
 - Diagram



E-R diagram corresponding to customers and loans.

17. Explain about Database Design Process **APRIL 2015**

Database Design for Banking Enterprise

Data Requirements

The major characteristics of the banking enterprise:

- The bank is organized into branches. Each branch is located in a particular city and is identified by a unique name. The bank monitors the assets of each branch.
- Bank customers are identified by their customer-id values. The bank stores each customer's name, and the street and city where the customer lives. Customer may have accounts and can take out loans. A customer may be associated with a particular banker, who may act as a loan officer or personal banker for that customer.

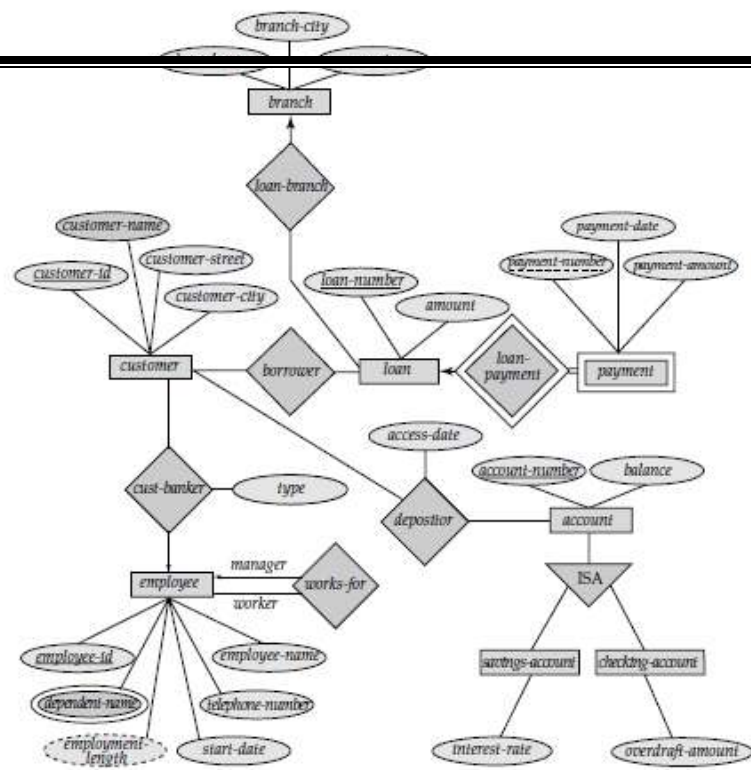
Entity Sets Designation

- The branch entity set, with attributes branch-name, branch-city, and assets.
- The customer entity set, with attributes customer-id, customer-name, customerstreet; and customer-city. A possible additional attribute is banker-name.

Relationship Sets Designation

- borrower, a many-to-many relationship set between customer and loan.
- loan-branch, a many-to-one relationship set that indicates in which branch a loan originated.

E-R Diagram



Reduction of an E-R Schema to Tables

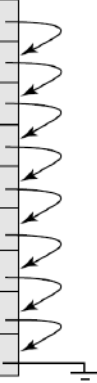
We have several of the possible ways of organizing records in files are:

- Heap file organization: Any record can be placed anywhere in the file where there is space for the record. There is no ordering of records. Typically, there is a single file for each relation.
- Sequential file organization: Records are stored in sequential order, according to the value of a “search key” of each record.
- Hashing file organization: A hash function is computed on some attribute of each record. The result of the hash function specifies in which block of the file the record should be placed.
- Generally, a separate file is used to store the records of each relation.
- However, in a clustering file organization, records of several different relations are stored in the same file; further related records of the different relations are stored in the same block, so that one I/O operation fetches related records from all the relations.

SEQUENTIAL FILE ORGANIZATION:

- A sequential file is designed for efficient processing of records in sorted order based on some search-key.
- A search key is any attribute or set of attributes; it need not be the primary key, or even a super key.
- To permit fast retrieval of records in search-key order, we chain together records by pointers. The pointer in each record points to the next record in search-key

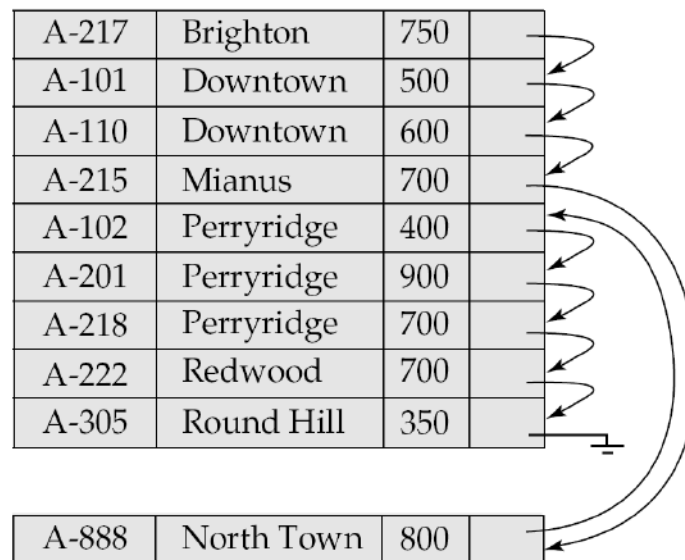
A-217	Brighton	750	
A-101	Downtown	500	
A-110	Downtown	600	
A-215	Mianus	700	
A-102	Perryridge	400	
A-201	Perryridge	900	
A-218	Perryridge	700	
A-222	Redwood	700	
A-305	Round Hill	350	



order,.

- A sequential file of account records taken from our banking example. In that example, the records are stored in search-key order, using branch name as the search key.
- It is difficult, however, to maintain physical sequential order as records are inserted and deleted, since it is costly to move many records as a result of a single insertion or deletion.

- We can manage deletion by using pointer chains, as we saw previously.
- For insertion, we apply the following rules:
- Locate the record in the file that comes before the record to be inserted in search-key order.
- If there is a free record (ie., space left after a deletion) within the same block as this record, insert the new record there. Otherwise, insert the new record in an overflow block. In either case, adjust the pointers so as to chain together the records in search-key



order.

Sequential file after an insertion.

- The file after the insertion of the record (NorthTown,A-888,800). The structure in the allows fast insertion of new records, but forces sequential file-processing applications to process records in an order that does not match the physical order of the records.
- If relatively few records need to be stored in overflow blocks, this approach works well.
- At this point, the file should be reorganized so that it is once again physically in sequential order. Such reorganizations are costly, and must be done during times when the system load is low..
- In the extreme case in which insertions rarely occur, it is possible always to keep the file in physically sorted order. In such a case, the pointer field is not needed.

CLUSTERING FILE ORGANIZATION:

- Many relational-database systems store each relation in a separate file, so that they can take full advantage of the file system, that the operating systems provides
- The tuples of a relation can be represented as fixed-length records and can be mapped to simple file structure.

- The simple implementation of a relational database system is well suited to low-cost database implementations as in, for example, embedded systems or portable devices.
- A simple file structure reduces the amount of code needed to implement the system.
- This simple approach to relational-database implementation becomes less satisfactory as the size of the database increases.

However, many large-scalar database systems do not rely directly on the underlying operating system for file management. Instead, one large operating-system file is allocated to the database system. The database system stores all relations in this one file, and manages the file itself.

To see the advantage of storing many relations in this one file, consider the following SQL query for the bank database:

```
select account-number, customer-name, customer-street, customer-city
from depositer, customer
where depositer.customer-name=customer.customer-name.
```

- This query computes a join of the depositer and customer relations.
- Thus, for each tuple of depositer, the system must locate the customer tuples with same value for customer-name.
- Ideally, these records will be located with the help of indices.
- In the worst case, each record will reside on a different block, forcing us to do one block read for each record required by the query.

<i>customer-name</i>	<i>account-number</i>
Hayes	A-102
Hayes	A-220
Hayes	A-503
Turner	A-305

DEPOSITOR and CUSTOMER Relation

Storage and File Structure

<i>customer-name</i>	<i>customer-street</i>	<i>customer-city</i>
Hayes	Main	Brooklyn
Turner	Putnam	Stamford

- The above table file structure designed for efficient execution of queries involving depositor & customer.

- The depositor tuples for each customer-name are stored near the customer tuple for the corresponding customer-name.
- This structure mixes together tuples of two relations, but allows for efficient processing of the join.
- When a tuple of the customer relation is read, the entire block contain that tuple is copied from disk into main memory.
- Since the depositor tuples are stored on the disk near the customer-tuple, the block containing the customer tuple contains tuples of the depositor relation needed to process the query. If a customer has so many accounts that the depositor records do not fit in one block, the remaining records appear on nearby blocks.
- A clustering file organization is a file organization, such as that illustrated in that stores related records of two or more relations in each block. Our use of clustering has enhanced processing of a particular join(depositor&cxustomer),but it results in slowing processing of other types of query.
- Requires more block accesses the than it did in the scheme under which we stored each relation in a separate file.

```
select account-number,customer-name,customer-street,customer-city
from depositer,customer
where depositer.customer-name=customer.customer-name
```

- when clustering is to be used depends on the types of query that the database designer believes to be most frequent. Careful use of clustering can produce significant performance gains in query processing.

DATA-DICTIONARY STORAGE:

A relational-database system needs to maintain data about the relations, such as the schema of the relations. The information is called the data dictionary, or system catalog. Among the types of information that the system must store are these:

Hayes	Main	Brooklyn
Hayes	A-102	
Hayes	A-220	
Hayes	A-503	
Turner	Putnam	Stamford
Turner	A-305	

Clustering the Structure

Clustering the Structure with Pointer

Hayes	Main	Brooklyn	
Hayes	A-102		
Hayes	A-220		
Hayes	A-503		
Turner	Putnam	Stamford	
Turner	A-305		



- Names of the relations
- Names of the attributes of each relation
- Domains and lengths of attributes
- Names of vies defined on the database, and definitions of those views
- Integrity constraints(For example, key constraints)

In addition, many systems keep the following data on users of the system:

- Names of authorized users
- Accounting information about users.
- Passwords or other information used to authenticate users

Further, the database may store statistical and descriptive data about the relations such as:

- Number of tuples in each relation
- Method of storage of each relation(for example, clustered or non-clustered)

The data dictionary may also note the storage organization (Sequential, hash or heap of relations, and the location where each relation is stored:

- If relations are stored in operating system files, the dictionary would note the names of the file(or files) containing each relation.
- If the database stores all relations in a single file, the dictionary may note the blocks containing records of each relation in a data structure such as a linked list.

In indices, we shall need to store information about each index on each of the relations:

- Name of the index
- Name of the relation being indexed
- Attributes on which the index is defined
- Type of index formed.

All this information constitutes, in effect, a miniature database. Some database systems store this information by using special-purpose data structure and code. It is generally-preferable to store the data about the database in the database itself.

The exact choice of how to represent system data by relations must be made by the system designers. One possible representation, with primary keys underlined is:

- ✓ Relation-metadata(relation-name, number of attributes, storage-organizational, location)
- ✓ Attribute-metadata(attribute-name, relation-name, domain-type, position, length)
- ✓ User-metadata(user-name, encrypted-password, group)
- ✓ Index-metadata(index-name, relation-name, index-type, index-attribute)
- ✓ View-metadata(view-name, definition)

19. What is meant by Hash function? Also discuss about dynamic hashing

Hash Functions {NOV 2015} (DEC2018)

1. A good hash function gives an average-case lookup that is a small constant, independent of the number of search keys.
2. We hope records are distributed uniformly among the buckets.
3. The worst hash function maps all keys to the same bucket.
4. The best hash function maps all keys to distinct addresses.
5. Ideally, distribution of keys to addresses is uniform and random.
6. Suppose we have 26 buckets, and map names beginning with i th letter of the alphabet to the i th bucket.
 - Problem: this does not give uniform distribution.
 - Many more names will be mapped to "A" than to "X".
 - Typical hash functions perform some operation on the internal binary machine representations of characters in a key.
 - For example, compute the sum, modulo of buckets, of the binary representations of characters of the search key.
 - See Figure 11.18, using this method for 10 buckets (assuming the i th character in the alphabet is represented by integer i).

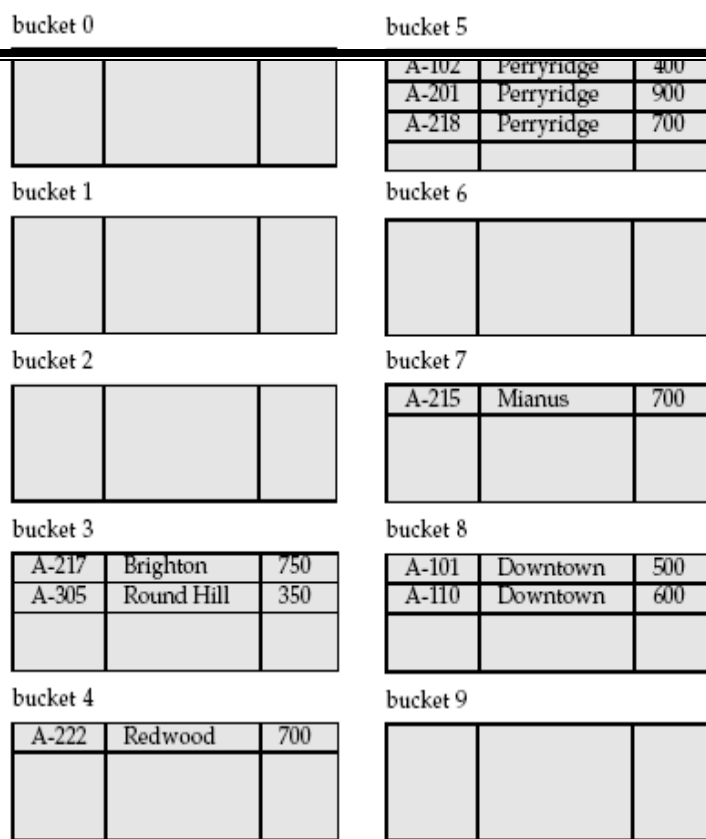


Figure 12.21 Hash organization of *account* file, with *branch-name* as the key.

Handling of bucket overflows

1. we handle bucket overflow by using *overflow buckets*.
2. All the overflow buckets of a given bucket are chained together in a linked list , as in fig 12.22

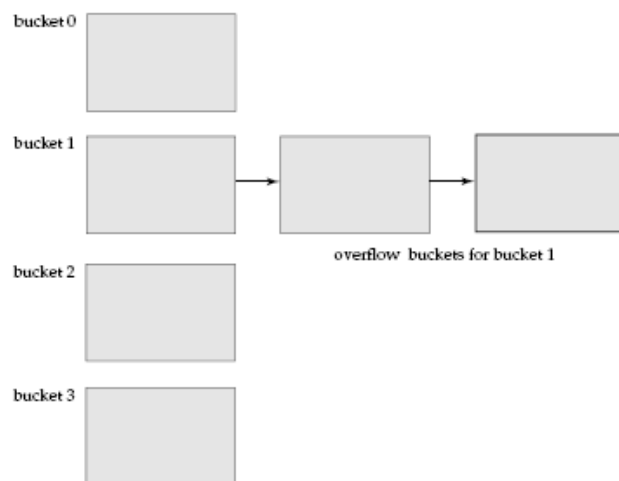


Figure 12.22 Overflow chaining in a hash structure.

3. **Open** hashing occurs where records are stored in different buckets. Compute the hash function and search the corresponding bucket to find a record.
4. **Closed** hashing occurs where all records are stored in **one** bucket. Hash function computes addresses within that bucket. (Deletions are difficult.) Not used much in database applications.

5. **Drawback to our approach:** Hash function must be chosen at implementation time.

- Number of buckets is fixed, but the database may grow.
- If number is too large, we waste space.
- If number is too small, we get too many "collisions", resulting in records of many search key values being in the same bucket.
- Choosing the number to be twice the number of search key values in the file gives a good space/performance tradeoff.

Hash Indices

1. A hash index organizes the search keys with their associated pointers into a hash file structure.
2. We apply a hash function on a search key to identify a bucket, and store the key and its associated pointers in the bucket (or in overflow buckets).
3. Strictly speaking, hash indices are only secondary index structures, since if a file itself is organized using hashing, there is no need for a separate hash index structure on it.

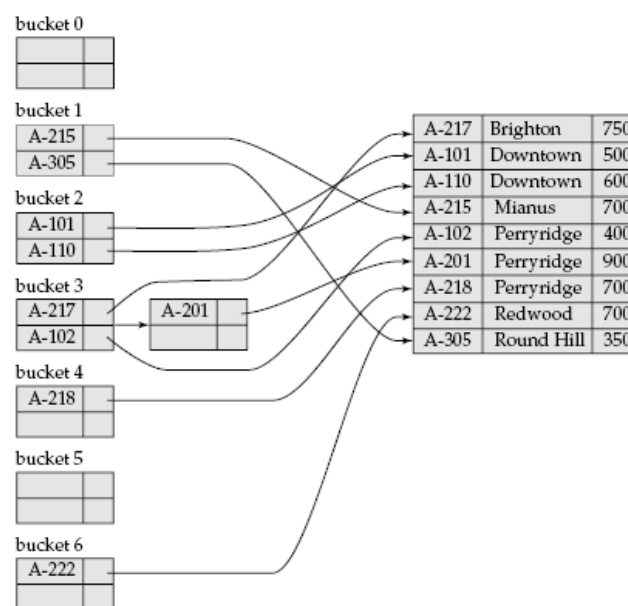


Figure 12.23 Hash index on search key account-number of account file.

DYNAMIC HASHING

1. As the database grows over time, we have three options:
 - Choose hash function based on current file size. Get performance degradation as file grows.
 - Choose hash function based on anticipated file size. Space is wasted initially.
 - Periodically re-organize hash structure as file grows. Requires selecting new hash function, recomputing all addresses and generating new bucket assignments. Costly, and shuts down database.

2. Some hashing techniques allow the hash function to be modified dynamically to accommodate the growth or shrinking of the database. These are called **dynamic hash functions**.

- **Extendable hashing** is one form of dynamic hashing.
- Extendable hashing splits and coalesces buckets as database size changes.
- This imposes some performance overhead, but space efficiency is maintained.
- As reorganization is on one bucket at a time, overhead is acceptably low.

3. How does it work?

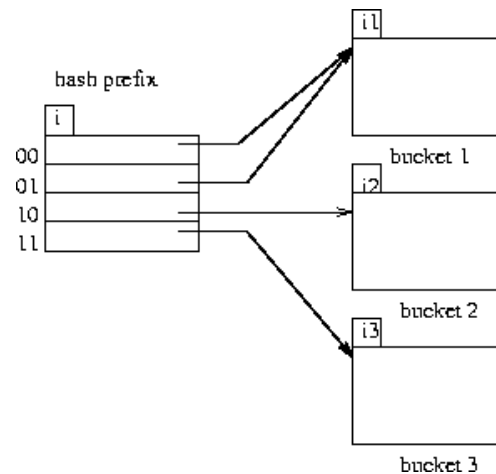


Figure 12.4 General extendable hash structure.

- We choose a hash function that is uniform and random that generates values over a relatively large range.
- Range is b -bit binary integers (typically $b=32$).
- 2^{32} is over 4 billion, so we don't generate that many buckets!
- Instead we create buckets on demand, and do not use all b bits of the hash initially.

$$0 \leq i \leq b$$

- The i bits are used as an offset into a table of bucket addresses.
- Value of i grows and shrinks with the database.
- Figure 12.4 shows an extendable hash structure.
- Note that the i appearing over the bucket address table tells how many bits are required to determine the correct bucket.
- It may be the case that several entries point to the same bucket.
- All such entries will have a common hash prefix, but the length of this prefix may be less than i .
- So we give each bucket an integer giving the length of the common hash prefix.

- The integer associated with bucket j is shown as i_j .
 - Number of bucket entries pointing to bucket j is then $2^{(i-i_j)}$.
4. To find the bucket containing search key value K_i :
- Compute $h(K_i)$.
 - Take the first i high order bits of $h(K_i)$.
 - Look at the corresponding table entry for this i -bit string.
 - Follow the bucket pointer in the table entry.
5. We now look at insertions in an extendable hashing scheme.
- Follow the same procedure for lookup, ending up in some bucket j .
 - If there is room in the bucket, insert information and insert record in the file.
 - If the bucket is full, we must split the bucket, and redistribute the records.
 - If bucket is split we may need to increase the number of bits we use in the hash.
6. Two cases exist:
1. If $i = i_j$, then only one entry in the bucket address table points to bucket j .
 - Then we need to increase the size of the bucket address table so that we can include pointers to the two buckets that result from splitting bucket j .
 - We increment i by one, thus considering more of the hash, and doubling the size of the bucket address table.
 - Each entry is replaced by two entries, each containing original value.
 - Now two entries in bucket address table point to bucket j .
 - We allocate a new bucket z , and set the second pointer to point to z .
 - Set i_j and i_z to i .
 - Rehash all records in bucket j which are put in either j or z .
 - Now insert new record.
 - It is remotely possible, but unlikely, that the new hash will still put all of the records in one bucket.
 - If so, split again and increment i again.
 2. If $i > i_j$, then more than one entry in the bucket address table points to bucket j .
 - Then we can split bucket j without increasing the size of the bucket address table
 - Note that all entries that point to bucket j correspond to hash prefixes that have the same value on the leftmost i_j bits.

- We allocate a new bucket z , and set i_z and i_z to the original i_j value plus 1.
 - Now adjust entries in the bucket address table that previously pointed to bucket j .
 - Leave the first half pointing to bucket j , and make the rest point to bucket z .
 - Rehash each record in bucket j as before.
 - Reattempt new insert.
7. Note that in both cases we only need to rehash records in bucket j .
 8. Deletion of records is similar. Buckets may have to be coalesced, and bucket address table may have to be halved.
 9. Insertion is illustrated for the example *deposit* file.

A-217	Brighton	750
A-101	Downtown	500
A-110	Downtown	600
A-215	Mianus	700
A-102	Perryridge	400
A-201	Perryridge	900
A-218	Perryridge	700
A-222	Redwood	700
A-305	Round Hill	350

Figure 12.25 Sample *account* file.

- 32-bit hash values on *bname* are shown in Figure

<i>branch-name</i>	$h(\text{branch-name})$
Brighton	0010 1101 1111 1011 0010 1100 0011 0000
Downtown	1010 0011 1010 0000 1100 0110 1001 1111
Mianus	1100 0111 1110 1101 1011 1111 0011 1010
Perryridge	1111 0001 0010 0100 1001 0011 0110 1101
Redwood	0011 0101 1010 0110 1100 1001 1110 1011
Round Hill	1101 1000 0011 1111 1001 1100 0000 0001

Figure 12.26 Hash function for *branch-name*.

- . An initial empty hash structure is shown in Figure



Figure 12.27 Initial extendable hash structure.

- We insert records one by one.
- We (unrealistically) assume that a bucket can only hold 2 records, in order to illustrate both situations described.

- As we insert the Perryridge and Round Hill records, this first bucket becomes full.
- When we insert the next record (Downtown), we must split the bucket.
- Since $i = i_0$, we need to increase the number of bits we use from the hash.
- We now use 1 bit, allowing us $2^1 = 2$ buckets.
- This makes us double the size of the bucket address table to two entries.
- We split the bucket, placing the records whose search key hash begins with 1 in the new bucket, and those with a 0 in the old bucket (Figure 11.23).
- Next we attempt to insert the Redwood record, and find it hashes to 1.
- That bucket is full, and $i = i_1$.
- So we must split that bucket, increasing the number of bits we must use to 2.
- This necessitates doubling the bucket address table again to four entries (Figure 12.28).

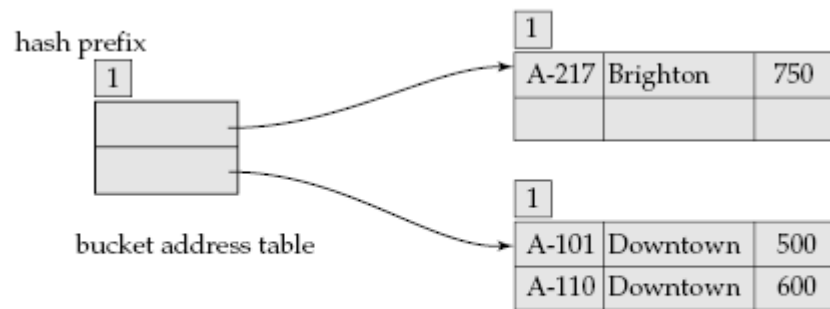


Figure 12.28 Hash structure after three insertions.

- We rehash the entries in the old bucket.
- We continue on for the deposit records of Figure 12.25, obtaining the extendable hash structure of Figure 12.31

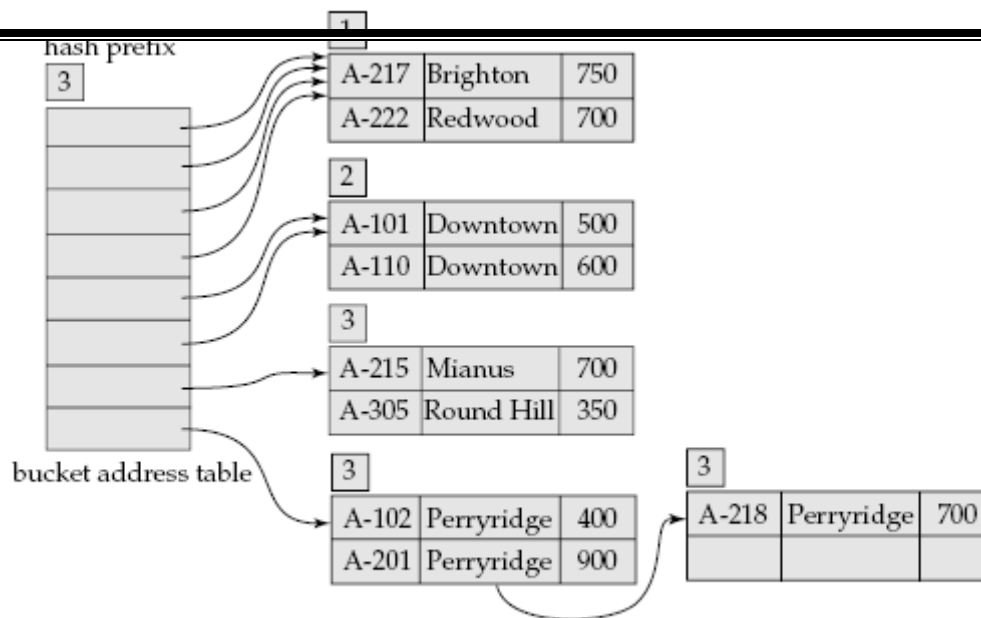


Figure 12.31 Extendable hash structure for the *account* file.

Advantages:

- Extendable hashing provides performance that does not degrade as the file grows.
- Minimal space overhead - no buckets need be reserved for future use. Bucket address table only contains one pointer for each hash value of current prefix length.

Disadvantages:

- Extra level of indirection in the bucket address table
- Added complexity

20 .Explain B-Tree index files in detail

(NOV 2010)

B+ TREE INDEX FILES

1. Primary disadvantage of index-sequential file organization is that performance degrades as the file grows. This can be remedied by costly re-organizations.
2. B +-tree file structure maintains its efficiency despite frequent insertions and deletions. It imposes some acceptable update and space overheads.
3. A B +-tree index is a *balanced tree* in which every path from the root to a leaf is of the same length.
4. Each nonleaf node in the tree must have between $\lceil n/2 \rceil$ and n children, where n is fixed for a particular tree.

Structure of a B+-Tree

1. A B +-tree index is a *multilevel* index but is structured differently from that of multi-level index sequential files.

A typical node (Figure 12.6) contains up to $n-1$ search key values $K_1, K_2, \dots, K_{n-1}$, and n pointers $P_1, P_2, \dots, P_n$. Search key values in a node are kept in sorted order.



Figure 12.6 Typical node of a B⁺-tree.

- For leaf nodes, P_i ($i = 1, \dots, n-1$) points to either a file record with search key value K_i , or a bucket of pointers to records with that search key value. Bucket structure is used if search key is not a primary key, and file is not sorted in search key order.

Pointer P_n (n th pointer in the leaf node) is used to chain leaf nodes together in linear order (search key order). This allows efficient **sequential** processing of the file.

The range of values in each **leaf** do not overlap.

- Non-leaf nodes form a multilevel index on leaf nodes.

A non-leaf node may hold up to n pointers and must hold $\lceil n/2 \rceil$ pointers. The number of pointers in a node is called the *fan-out* of the node.

Consider a node containing m pointers. Pointer P_i ($i = 2, \dots, m$) points to a subtree containing search key values $\geq K_{i-1}$ and $< K_i$. Pointer P_m points to a subtree containing search key values $\geq K_{m-1}$. Pointer P_1 points to a subtree containing search key values $< K_1$.

- Figures 11.7 (textbook Fig. 11.8) and textbook Fig. 11.9 show B⁺-trees for the *deposit* file with $n=3$ and $n=5$.

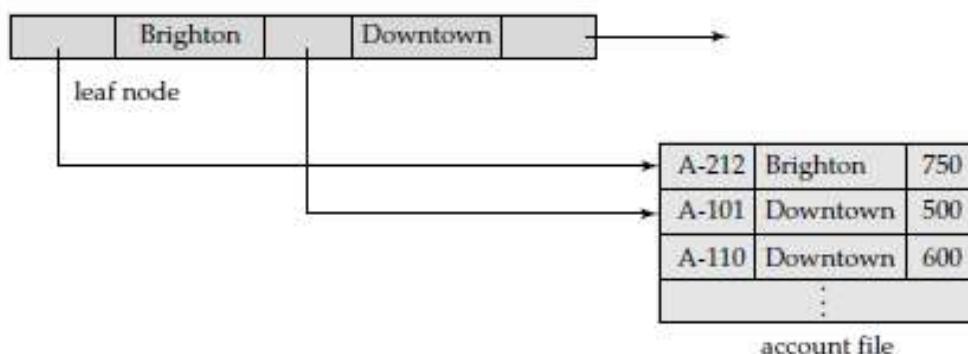


Figure 12.7 A leaf node for *account* B⁺-tree index ($n = 3$).

Queries on B⁺-Trees

- Suppose we want to find all records with a search key value of k .
 - Examine the root node and find the smallest search key value $K_i > k$.

- Follow pointer P_i to another node.
 - If $k < K_i$, follow pointer P_i .
 - Otherwise, find the appropriate pointer to follow.
 - Continue down through non-leaf nodes, looking for smallest search key value $> k$ and following the corresponding pointer.
 - Eventually we arrive at a leaf node, where pointer will point to the desired record or bucket.
2. In processing a query, we traverse a path from the root to a leaf node. If there are K search key values in the file, this path is no longer than $\log_{\lceil n/2 \rceil}(K)$.

This means that the path is not long, even in large files. For a 4k byte disk block with a search-key size of 12 bytes and a disk pointer of 8 bytes, n is around 200. If $n = 100$, a look-up of 1 million search-key values may take $\log_{50}(1,000,000) = 4$ nodes to be accessed. Since root is usually in the buffer, so typically it takes only 3 or fewer disk reads.

Updates on B+-Trees

1. Insertions and Deletions:

Insertion and **deletion** are more complicated, as they may require splitting or combining nodes to keep the tree balanced. If splitting or combining are not required, insertion works as follows:

- Find leaf node where search key value should appear.
- If value is present, add new record to the bucket.
- If value is not present, insert value in leaf node (so that search keys are still in order).
- Create a new bucket and insert the new record.

If splitting or combining are not required, deletion works as follows:

- Deletion: Find record to be deleted, and remove it from the bucket.
- If bucket is now empty, remove search key value from leaf node.

2. Insertions Causing Splitting:

When insertion causes a leaf node to be too large, we **split** that node. In Figure 11.8, assume we wish to insert a record with a *bname* value of "Clearview".

- There is no room for it in the leaf node where it should appear.
- We now have n values (the $n-1$ search key values plus the new one we wish to insert).

- We put the first $\lceil n/2 \rceil$ values in the existing node, and the remainder into a new node.
- Figure 11.10 shows the result.
- The new node must be inserted into the B +-tree.
- We also need to update search key values for the parent (or higher) nodes of the split leaf node. (Except if the new node is the leftmost one)
- Order must be preserved among the search key values in each node.
- If the parent was already full, it will have to be split.
- When a non-leaf node is split, the children are divided among the two new nodes.
- In the worst case, splits may be required all the way up to the root. (If the root is split, the tree becomes one level deeper.)
- **Note:** when we start a B +-tree, we begin with a single node that is both the root and a single leaf. When it gets full and another insertion occurs, we split it into two leaf nodes, requiring a new root.

3. Deletions Causing Combining:

Deleting records may cause tree nodes to contain too few pointers. Then we must combine nodes.

- If we wish to delete "Downtown" from the B +-tree of Figure 11.11, this occurs.
- In this case, the leaf node is empty and must be deleted.
- If we wish to delete "Perryridge" from the B +-tree of Figure 11.11, the parent is left with only one pointer, and must be coalesced with a sibling node.
- Sometimes higher-level nodes must also be coalesced.
- If the root becomes empty as a result, the tree is one level less deep (Figure 11.13).
- Sometimes the pointers must be redistributed to keep the tree balanced.
- Deleting "Perryridge" from Figure 11.11 produces Figure 11.14.

4. To summarize:

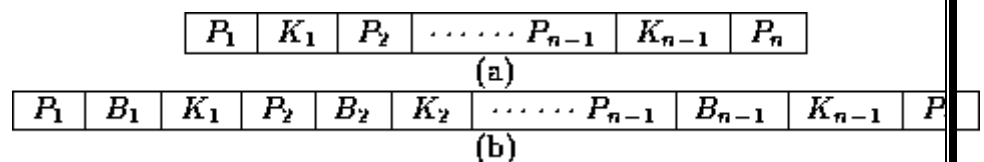
- Insertion and deletion are complicated, but require relatively few operations.
- Number of operations required for insertion and deletion is proportional to logarithm of number of search keys.
- B +-trees are fast as index structures for database.

B+-Tree File Organization

1. The B+-tree structure is used not only as an index but also as an organizer for records into a file.
2. In a B+-tree file organization, the leaf nodes of the tree store records instead of storing pointers to records, as shown in Fig. 11.17.
3. Since records are usually larger than pointers, the maximum number of records that can be stored in a leaf node is less than the maximum number of pointers in a non leaf node.
4. However, the leaf node are still required to be at least half full.
5. Insertion and deletion from a B+-tree file organization are handled in the same way as that in a B+-tree index.
6. When a B+-tree is used for file organization, space utilization is particularly important. We can improve the space utilization by involving more sibling nodes in redistribution during splits and merges.
7. In general, if m nodes are involved in redistribution, each node can be guaranteed to contain at least $\lceil (m-1)n/m \rceil$ entries. However, the cost of update becomes higher as more siblings are involved in redistribution.

B-TREE INDEX FILES

1. B-tree indices are similar to B+-tree indices.
 - Difference is that B-tree eliminates the redundant storage of search key values.
 - In B+-tree, some search key values appear twice.
 - A corresponding B-tree of Figure 11.18 allows search key values to appear only once.
 - Thus we can store the index in less space.



Leaf and non leaf node of a B-tree.

2. Advantages:

- Generally, the structural simplicity of B+-tree is preferred.
- Lack of redundant storage (but only marginally different).
- Some searches are faster (key may be in non-leaf node).

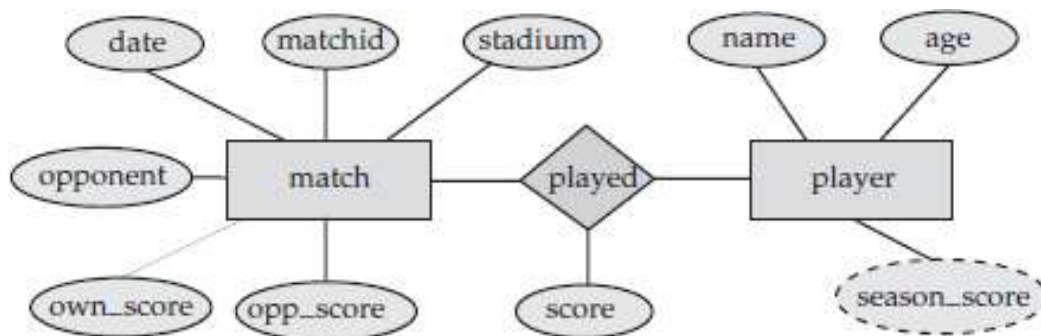
3. Disadvantages:

- Leaf and non-leaf nodes are of different size (complicates storage)

Deletion may occur in a non-leaf node (more complicated)

21. Design an E-R diagram for keeping track of exploits of your favorite sports team. You should store the matches played, the scores in each match, the players in each match, and individual player statistics for each match. Summary statistics should be modeled as derived attributes. [April 2017]

Answer:



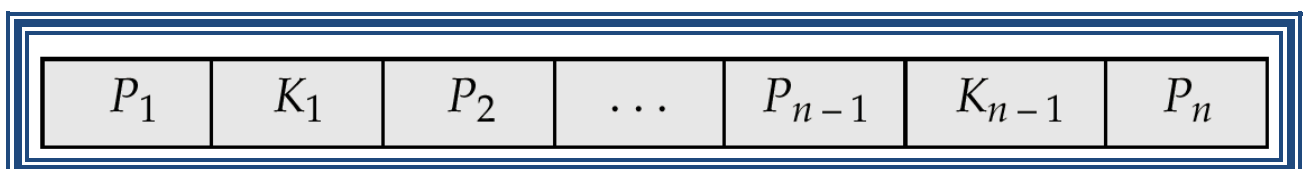
22. Explain the distinction between closed and open hashing. Discuss the relative merits of each technique in database applications. [April 2017]

A hash table is where data storage for a key-value pair is done by generating an index using a hash function. Open Hashing (aka Separate chaining) is simpler to implement, and more efficient for large records or sparse tables. Closed Hashing (aka Open Addressing) is more complex but can be more efficient, especially for small data records.

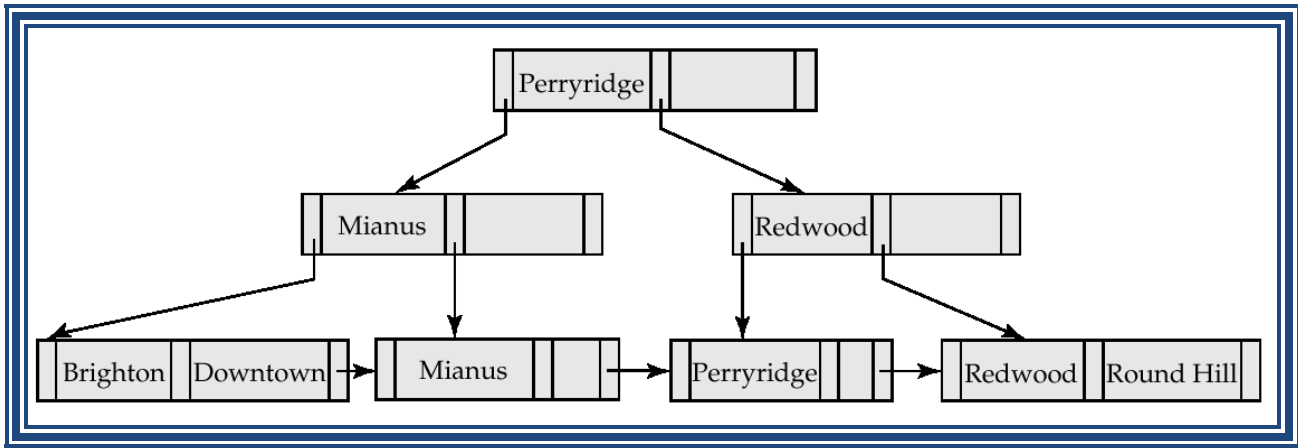
23. Explain about B+Tree index files (NOV 2012)

The Non leaf nodes form a multi-level sparse index on the leaf nodes. For a non-leaf node with m pointers:

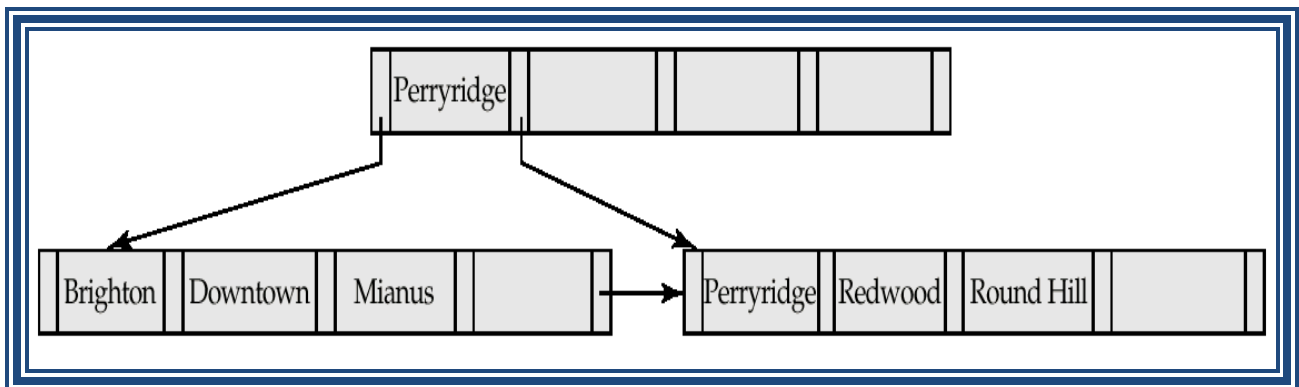
- All the search-keys in the sub tree to which P_1 points are less than K_1
- For $2 \leq i \leq n - 1$, all the search-keys in the sub tree to which P_i points have values greater than or equal to K_{i-1} and less than K_{m-1}



- Leaf nodes must have between 2 and 4 values ($\lceil (n-1)/2 \rceil$ and $n-1$, with $n=5$).
- Non-leaf nodes other than root must have between 3 and 5 children ($\lceil n/2 \rceil$ and n with $n=5$).
- Root must have at least 2 children.



B⁺-tree for *account* file ($n = 3$)



B⁺-tree for *account* file ($n = 5$)

A B⁺-tree is one in a family of multi-way search trees (others are: B-tree, 2-3-4 Tree, B^{*}-Trees). These trees were first proposed by Bayer and McCreight in 1972 and in just a few years had replaced almost all large file access methods other than hashing. These multi-way balanced search trees are now the standard file organization for applications requiring insertion, deletion, and key range searches. They have the following advantages:

- B-trees are always height balanced, with all leaf nodes at the same level
- Update and search operations affect only a few disk pages, so performance is good.
- B-trees keep related records on the same disk page, which takes advantage of locality of reference.
- B-trees guarantee that every node in the tree will be full at least to a certain minimum percentage. This improves space efficiency while reducing the typical number of disk fetches necessary during a search or update operation over many thousands of records.

B⁺-tree Structure

A B⁺-tree in certain aspects is a generalization of a binary search tree (BST). The main difference is that nodes of a B⁺-tree will point to many children nodes rather than being limited to only two. Since our goal is to minimize disk accesses whenever we are trying to locate records, we want to

make the height of the multi-way search tree as small as possible. This goal is achieved by having the tree branch in large amounts at each node.

A B⁺-tree of order m is a tree where each internal node contains up to m branches (children nodes) and thus store up to $m-1$ search key values -- in a BST, only one key value is needed since there are just two children nodes that an internal node can have. m is also known as the branching factor or the fanout of the tree.

1. The B⁺-tree stores records (or pointers to actual records) only at the leaf nodes, which are all found at the same level in the tree, so the tree is always height balanced.
2. All internal nodes, except the root, have between $\lceil m/2 \rceil$ and m children
3. The root is either a leaf or has at least two children.
4. Internal nodes store search key values, and are used only as placeholders to guide the search. The number of search key values in each internal node is one less than the number of its non-empty children, and these keys partition the keys in the children in the fashion of a search tree. The keys are stored in non-decreasing order (i.e. sorted in lexicographical order).
5. Depending on the size of a record as compared to the size of a key, a leaf node in a B⁺-tree of order m may store more or less than m records. Typically this is based on the size of a disk block, the size of a record pointer, etc. The leaf pages must store enough records to remain at least half full.
6. The leaf nodes of a B⁺-tree are linked together to form a linked list. This is done so that the records can be retrieved sequentially without accessing the B⁺-tree index. This also supports fast processing of range-search queries as will be described later.

B⁺-tree operations

To understand the B⁺-tree operations more clearly, assume, without loss of generality, that there is a table whose primary is a single attribute and that it has a B⁺-tree index organized on the PK attribute of the table.

Searching for records that satisfy a simple condition

To retrieve records, queries are written with conditions that describe the values that the desired records are to have. The most basic search on a table to retrieve a single record given its PK value K .

Search in a B⁺-tree is an alternating two-step process, beginning with the root node of the B⁺-tree. Say that the search is for the record with key value K -- there can only be one record because we assume that the index is built on the PK attribute of the table.

1. Perform a binary search on the search key values in the current node -- recall that the search key values in a node are sorted and that the search starts with the root of the tree. We want to find the key K_i such that $K_i \leq K < K_{i+1}$.
2. If the current node is an internal node, follow the proper branch associated with the key K_i by loading the disk page corresponding to the node and repeat the search process at that node.
3. If the current node is a leaf, then:

- a. If $K=K_i$, then the record exists in the table and we can return the record associated

with K_i

- b. Otherwise, K is not found among the search key values at the leaf, we report that there is no record in the table with the value K .

Inserting into a B⁺-tree

Insertion in a B⁺-tree is similar to inserting into other search trees, a new record is always inserted at one of the leaf nodes. The complexity added is that insertion could overflow a leaf node that is already full. When such overflow situations occur a brand new leaf node is added to the B⁺-tree at the same level as the other leaf nodes. The steps to insert into a B⁺-tree are:

1. Follow the path that is traversed as if a Search is being performed on the key of the new record to be inserted.
 2. The leaf page L that is reached is the node where the new record is to be indexed.
 3. If L is not full then an index entry is created that includes the search key value of the new row and a reference to where new row is in the data file. We are done; this is the easy case!
 4. If L is full, then a new leaf node L_{new} is introduced to the B⁺-tree as a right sibling of L . The keys in L along with the an index entry for the new record are distributed evenly among L and L_{new} . L_{new} is inserted in the linked list of leaf nodes just to the right of L . We must now link L_{new} to the tree and since L_{new} is to be a sibling of L , it will then be pointed to by the parent of L . The smallest key value of L_{new} is **copied** and inserted into the parent of L -- which will also be the parent of L_{new} . This entire step is known as commonly referred to as a **split of a leaf node**.
- a. If the parent P of L is full, then it is split in turn. However, this **split of an internal node** is a bit different. The search key values of P and the new inserted key must still be distributed evenly among P and the new page introduced as a sibling of P . In this split, however, the **middle key is moved to the node above** -- note, that unlike splitting a leaf node where the middle key is copied and inserted into the parent, when you split an internal node the middle key is removed from the node being split and inserted into the parent node. This splitting of nodes may continue upwards on the tree.
- b. When a key is added to a full root, then the root splits into two and the middle key is promoted to become the new root. This is the only way for a B⁺-tree to increase in height -- when split cascades the entire height of the tree from the leaf to the root.

Deletion

Deletion from a B⁺-tree again needs to be sure to maintain the property that all nodes must be at least half full. The complexity added is that deletion could underflow a leaf node that has only the minimum number of entries allowed. When such underflow situations take place, *adjacent sibling nodes are examined*; if one of them has more than the minimum entries required then, some of its entries are taken from it to prevent a node from "under-flowing". Otherwise, if both adjacent sibling nodes are also at their minimum, then two of these nodes are merged into a single node. The steps to delete from a B⁺-tree are:

1. Perform the search process on the key of the record to be deleted. This search will end at a

leaf L.

2. If the leaf L contains more than the minimum number of elements (more than $m/2 - 1$), then the index entry for the record to be removed can be safely deleted from the leaf with no further action.
3. If the leaf contains the minimum number of entries, then the deleted entry is replaced with another entry that can take its place while maintaining the correct order. To find such entries, we inspect the two sibling leaf nodes L_{left} and L_{right} adjacent to L -- at most one of these may not exist.
 - a. If one of these leaf nodes has more than the minimum number of entries, then enough records are transferred from this sibling so that both nodes have the same number of records. This is a heuristic and is done to delay a future underflow as long as possible; otherwise, only one entry need be transferred. The placeholder key value of the parent node may need to be revised.
 - b. If both L_{left} and L_{right} have only the minimum number of entries, then L gives its records to one of its siblings and it is removed from the tree. The new leaf will contain no more than the maximum number of entries allowed. This merge process combines two subtrees of the parent, so the separating entry at the parent needs to be removed -- this may in turn cause the parent node to underflow; such an underflow is handled the same way that an underflow of a leaf node.
 - c. If the last two children of the root merge together into one node, then this merged node becomes the new root and the tree loses a level.

Range Search

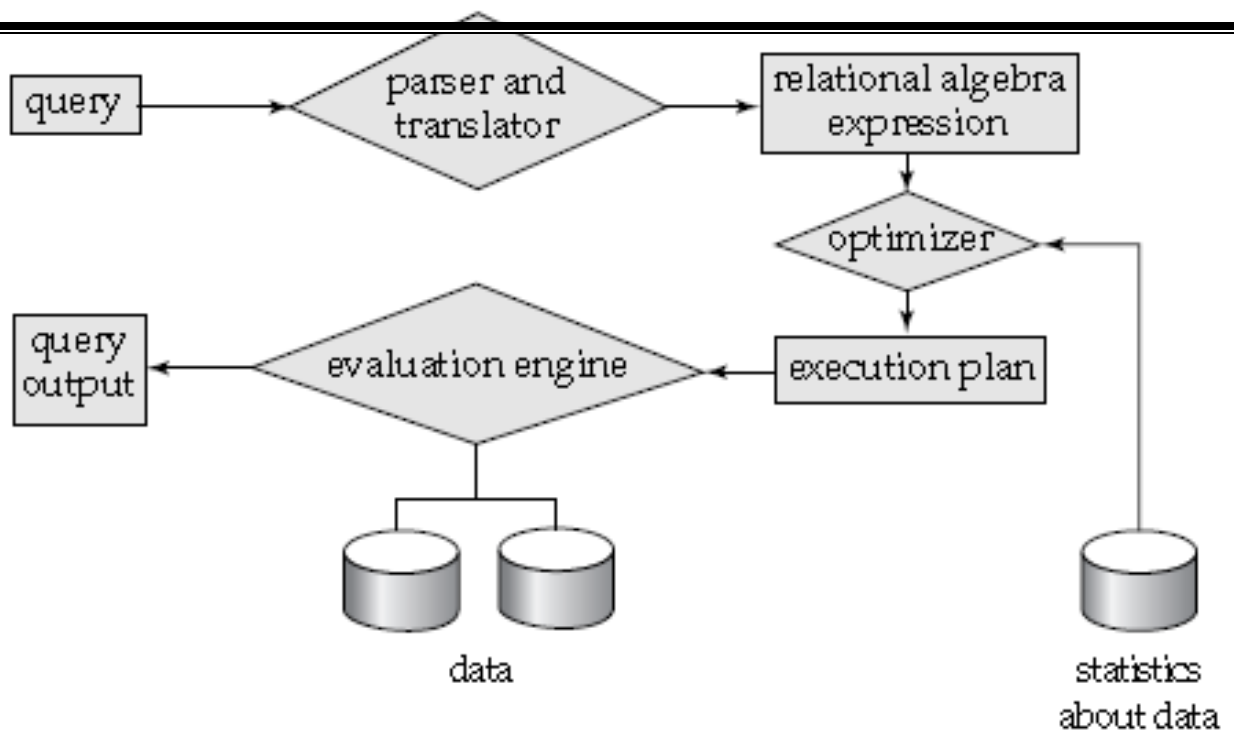
It is common to request records from a table that fall in a specified range. For example, we may want to retrieve all employees making salaries between \$50,000 and \$60,000. B⁺-trees are exceptionally good for range queries. Once the first record in the range has been found using the search algorithm described above, the rest of the records in the range can be found by sequential processing the remaining records in the leaf node, and then continuing down (actually right of the current leaf node) the linked list of leaf nodes as far as necessary. Once a record is found that has a search key value above the upper bound of the requested range, then the search completes. Note that to use a B⁺-tree index to retrieve the employees described above, the B⁺-tree index must exist that has been organized using the salary attribute of the employees table.

24.Explain Query Processing in detail.(Apr 2014)

Basic steps in query processing.

The steps involved in processing a query are

1. Parsing and translation
2. Optimization
3. Evaluation



The terms annotations and query evaluation primitive.

Annotations may state the algorithm to be used for a specific operation, or the particular index or indices to use. A relational-algebra operation annotated with instructions on how to evaluate it is called an **evaluation primitive**.

Query execution plan and query execution engine

A sequence of primitive operations that can be used to evaluate a query is a **query execution plan** or **query-evaluation plan**. Figure illustrates an evaluation plan in which a particular index is specified for the selection operation. The **query-execution engine** takes a query-evaluation plan, executes that plan, and returns the answers to the query.

25.Describe Indexing and Hashing.(Nov 2014) [DEC 2016] (DEC2018)

Indexing and Hashing

Hash tables are often used to implement associative arrays, sets and caches. Like arrays, hash tables provide constant-time $O(1)$ lookup on average, regardless of the number of items in the table. While theoretically the worst-case lookup time can be as bad as $O(n)$, this is for practical purposes statistically impossible unless the hash function is poorly designed or unless the set of keys is maliciously chosen with the given hash function in mind.

Compared to other associative array data structures, hash tables are most useful when large numbers of records are to be stored, especially if the size of the data set can be predicted.

Hash tables may be used as in-memory data structures. Hash tables may also be adopted

for use with persistent data structures; database indexes sometimes use disk-based data structures based on hash tables, although balanced trees are more popular.

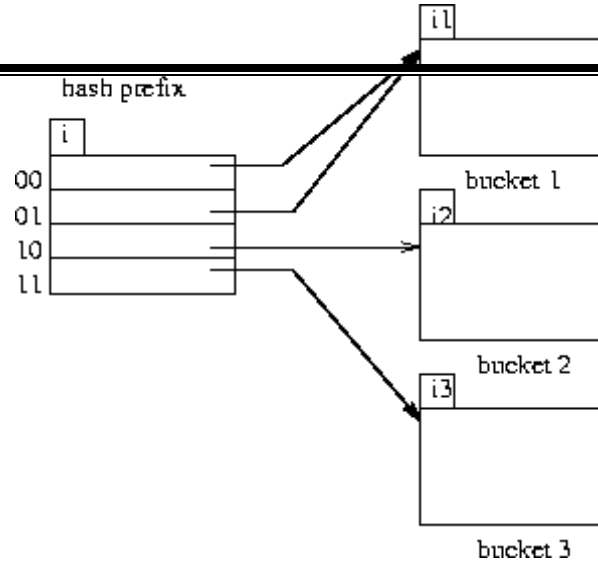
Static Hashing

Hash File Organization

1. **Hashing** involves computing the address of a data item by computing a function on the search key value.
2. A **hash function h** is a function from the set of all search key values K to the set of all bucket addresses B .
 - We choose a number of buckets to correspond to the number of search key values we will have stored in the database.
 - To perform a lookup on a search key value K_i , we compute $h(K_i)$, and search the bucket with that address.
 - If two search keys i and j map to the same address, because $h(K_i) = h(K_j)$, then the bucket at the address obtained will contain records with both search key values.
 - In this case we will have to check the search key value of every record in the bucket to get the ones we want.
 - Insertion and deletion are simple.

Dynamic Hashing

1. As the database grows over time, we have three options:
 - Choose hash function based on current file size. Get performance degradation as file grows.
 - Choose hash function based on anticipated file size. Space is wasted initially.
 - Periodically re-organize hash structure as file grows. Requires selecting new hash function, recomputing all addresses and generating new bucket assignments. Costly, and shuts down database.
2. Some hashing techniques allow the hash function to be modified dynamically to accommodate the growth or shrinking of the database. These are called **dynamic hash functions**.
 - **Extendable hashing** is one form of dynamic hashing.
 - Extendable hashing splits and coalesces buckets as database size changes.
 - This imposes some performance overhead, but space efficiency is maintained.
 - As reorganization is on one bucket at a time, overhead is acceptably low.



Index Definition in SQL

1. Some SQL implementations includes data definition commands to create and drop indices. The IBM SAA-SQL commands are

An index is created by

aaaaaaaaaaaaa~**create index** <index-name> **on** *r* (<attribute-list>)

the attribute list is the list of attributes in relation *r* that form the search key for the index.

To create an index on *bname* for the *branch* relation: aaaaaaaaaaaaa~**create index** *b-index* **on** *branch* (*bname*)

If the search key is a candidate key, we add the word **unique** to the definition:

aaaaaaaaaaaaa~**create unique index** *b-index* **on** *branch* (*bname*)

If *bname* is not a candidate key, an error message will appear.

If the index creation succeeds, any attempt to insert a tuple violating this requirement will fail.

The **unique** keyword is redundant if primary keys have been defined with integrity constraints already.

2. To remove an index, the command is
3. aaaaaaaaaaaaa~**drop index** <index-name>

Multiple key Access

1. For some queries, it is advantageous to use multiple indices if they exist.
2. If there are two indices on deposit, one on bname and one on cname, then suppose we have a query like
3. aaaaaaaaaaaaa~**select** balance
4. **from** deposit
5. **where** bname = ``Perryridge" **and** balance = 1000
6. There are 3 possible strategies to process this query:
 - Use the index on bname to find all records pertaining to Perryridge branch. Examine them to see if balance = 1000
 - Use the index on balance to find all records pertaining to Williams. Examine them to see if bname = ``Perryridge".
 - Use index on bname to find pointers to records pertaining to Perryridge branch. Use index on balance to find pointers to records pertaining to 1000. Take the **intersection** of these two sets of pointers.
7. The third strategy takes advantage of the existence of multiple indices. This may still not work well if
 - There are a large number of Perryridge records **AND**
 - There are a large number of 1000 records **AND**
 - Only a small number of records pertain to both Perryridge and 1000.
8. To speed up multiple search key queries special structures can be maintained.

26.Describe Query Optimization in detail. (Nov 2014)

Query optimization is the process of selecting the most efficient query-evaluation plan from among the many strategies usually possible for processing a given query, especially if the query is complex.

Query optimization is a function of many relational database management systems. The **query optimizer** attempts to determine the most efficient way to execute a given query by considering the possible query plans.

Generally, the query optimizer cannot be accessed directly by users: once queries are submitted to database server, and parsed by the parser, they are then passed to the query optimizer where optimization occurs. However, some database engines allow guiding the query optimizer with hints.

A query is a request for information from a database. It can be as simple as "finding the address of a person with SS# 123-45-6789," or more complex like "finding the average salary of all the

employed married men in California between the ages 30 to 39, that earn less than their wives."

Queries results are generated by accessing relevant database data and manipulating it in a way that yields the requested information. Since database structures are complex, in most cases, and especially for not-very-simple queries, the needed data for a query can be collected from a database by accessing it in different ways, through different data-structures, and in different orders. Each different way typically requires different processing time. Processing times of the same query may have large variance, from a fraction of a second to hours, depending on the way selected. The purpose of query optimization, which is an automated process, is to find the way to process a given query in minimum time. The large possible variance in time justifies performing query optimization, though finding the exact optimal way to execute a query, among all possibilities, is typically very complex, time consuming by itself, may be too costly, and often practically impossible. Thus query optimization typically tries to approximate the optimum by comparing several common-sense alternatives to provide in a reasonable time a "good enough" plan which typically does not deviate much from the best possible result.

27.Explain about Indexing concept with respect to database system. (APRIL 2015)[apr2018]

A search key is any attribute or set of attribute , it need not be the primary key or a even a super key for easy access is called Indexing.

Types of Indexing: (5 Marks)

Ordered indices: Based on a sorted ordering of the values.

Hash indices: Based on a uniform distribution of values across a range of bucket. The bucket to which a value is assigned is determined by a function, called a hash function.

An index record appears for every search key value in the file. **(6 Marks)**

In a dense clustering index, the index record contains search key value and a pointer to the first data record with that search key value.

An index record appears for only some of the search key values in a file. Such a

Indices with two or more levels are called multilevel indices, multilevel indices are closely related to the tree structures such as binary tree used for memory indexing.

UNIVERSITY QUESTIONS

1. Explain Authorization in SQL with example-Refer First unit notes**11 marks (APRIL 2011, (Question Number:5))**.
2. Write about ER Diagrams(11 marks)” Refer First unit notes”**11 marks APRIL 2012,NOV 2012,APR 2014, NOV 2015,NOV 2017(DEC2018) (MAY 2019)(Question Number:15)**
3. Explain any two normal forms in detail. **11 marks) APRIL 2012, (Question Number:10).**
4. Explain Decomposition using Functional Dependencies.(**11 marks) NOV 2012, (Question Number:10).**
5. Explain 3 NF and 4 NF with examples (**11 marks) APR 2014, (Question Number:13,14).**
6. Explain the Fundamentals of database design and ER model. **NOV 2014, (Question Number:9).**
7. Explain in details about application design and development. **Nov2014, (Question Number:15).**
8. Discuss the various concepts of ER Model with an example. **Apr 2015, (Question Number:16).**
9. Explain about Database Design Process. **Apr 2015, (Question Number:17).**
10. Explain organization of records in files structure in detail. **Apr 2011, (Question Number:18).**
11. Explain B-Tree index files in detail. **NOV 2010, (Question Number:20)**
12. What is meant by Hash function? Also discuss about dynamic hashing
Hash Functions {**NOV 2015](Page 66) (Question Number:19)**
13. Explain Extended E-R Features [**DEC 2016]**

- 14. Design an E-R diagram for keeping track of exploits of your favorite sports team. You should store the matches played, the scores in each match, the players in each match, and individual player statistics for each match. Summary statistics should be modeled as derived attributes. [April 2017] (Question Number:21)**
- 15. Explain the distinction between closed and open hashing. Discuss the relative merits of each technique in database applications. [April 2017] (Question Number:22)**
- 16. Explain about B+Tree index files (Nov 2012)[Question No. 10]**
- 17. Explain Query processing in detail. (April 2014) [Question No. 11]**
- 18. Describe Indexing and Hashing.(Nov 2014) [NOV 2018][Question No. 12]**
- 19. Describe Query Optimization in detail. (Nov 2014) [Question No. 13] [DEC 2016]**
- 20. Explain about Indexing concept with respect to database system. (Apr 2015) [Question No. 14]**
- 21.Explain Static and dynamic hashing [NOV 2017] (MAY 2019)**
- 22. Explain Bitmap Indices [NOV 2017]**